

Digitalización, estructuración y preservación lingüística de gramáticas mayas minorizadas mediante inteligencia artificial multimodal



Máster en Inteligencia Artificial

Trabajo Fin de Máster

Autor:

Elías Alfonso Carrasco Guerrero

Tutor:

Juan Antonio Pérez Ortiz

Setiembre 2026



Universitat d'Alacant
Universidad de Alicante

Digitalización, estructuración y preservación lingüística de gramáticas mayas minorizadas mediante inteligencia artificial multimodal

Automatización de la preservación de lenguas minorizadas: un enfoque basado en modelos de lenguaje y marcado XML

Autor

Elías Alfonso Carrasco Guerrero

Directores

Juan Antonio Pérez Ortiz

Departamento de Lenguajes y Sistemas Informáticos



MÁSTER EN INTELIGENCIA ARTIFICIAL



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

Alicante, 15 de junio de 2026

Resumen

La aplicación desarrollada para este Trabajo de Fin de Máster consiste en un software modular programado en Python que se ejecuta desde la terminal de comandos. Su función principal es automatizar la extracción de texto y la comprensión visual de páginas en formato PDF pertenecientes a gramáticas de lenguas mayas. El sistema inyecta ejemplos estructurados de entrada y salida (few-shot) junto a un prompt especializado directamente en la memoria de la API de Gemini, logrando que el modelo de Inteligencia Artificial entienda la disposición tipográfica original y devuelva la información totalmente organizada en un archivo XML con etiquetas semánticas personalizadas (como `¡morpheme.analysis!` o `¡syntactic.analysis!`). Posteriormente, el pipeline procesa este archivo XML para generar de forma automática un documento en LaTeX que se compila para reconstruir la obra original en un nuevo archivo PDF con un acabado editorial profesional. El objetivo último es transformar "papel digital. estático en un conjunto de datos estructurados de alta precisión (Gold Standard) que sirva como combustible indispensable para que, en el futuro, otras Inteligencias Artificiales puedan entrenarse y aprender idiomas minorizados de bajos recursos computacionales. Este Trabajo de Fin de Máster aborda la problemática de la digitalización y estructuración de gramáticas normativas de lenguas mayas minorizadas (Kaqchikel, K'iche', Mam y Q'anjob'al), cuyos documentos originales presentan estructuras tipográficas complejas que dificultan el procesamiento mediante técnicas de OCR convencionales. Se propone y desarrolla un sistema automatizado basado en Modelos de Lenguaje de Gran Tamaño (LLMs) a través de la API de Gemini, integrado en un pipeline modular desarrollado en Python. Con este flujo se consigue mitigar la pérdida de información semántica y espacial inherente a los métodos planos tradicionales, salvaguardando recursos científicos de alto valor como las glosas interlineales. La metodología se fundamenta en la transformación de documentos PDF en datos estructurados bajo el estándar XML, garantizando la integridad jerárquica mediante la definición de un Documento de Definición de Tipo (DTD) y el desarrollo de módulos específicos de corrección estructural (`xml.corrector.py`). El núcleo de la investigación se centra en un estudio empírico sobre el parámetro de temperatura del modelo, evaluando su impacto en la precisión y estabilidad del sistema bajo una estricta economía de tokens impuesta por las limitaciones de cuota del entorno operativo. Los resultados experimentales demuestran la existencia de un "punto dulce" (sweet spot) en temperaturas de 0.1 y 0.25, donde se alcanzan precisiones superiores al 90

Palabras clave: Inteligencia Artificial, LLM, Gemini API, Lenguas Mayas, Digitalización, XML, Procesamiento de Lenguaje Natural, OCR Estructurado.

Motivación y justificación

La preservación de la diversidad lingüística constituye uno de los desafíos más críticos en la era de la información. Las lenguas minorizadas, en particular aquellas pertenecientes a la familia maya como el Kaqchikel, el K'iche', el Mam y el Q'anjob'al, representan un patrimonio cultural de valor incalculable. Sin embargo, su supervivencia no sólo depende de su uso oral, sino también de la disponibilidad y accesibilidad de recursos académicos y normativos en formatos digitales modernos. En este contexto, la Academia de Lenguas Mayas de Guatemala (ALMG) desempeña un papel crucial como entidad rectora en la promoción y estandarización de estos idiomas. A través de su repositorio institucional de documentos (disponible en <https://almg.org.gt/documentos/>), la ALMG ha realizado un esfuerzo significativo por poner a disposición del público gramáticas normativas y textos de referencia, tal y como se muestra en la figura 1. No obstante, gran parte de este conocimiento se encuentra consignado en archivos PDF que, si bien facilitan la difusión, actúan esencialmente como "papel digital": una representación estática de la información que carece de estructura semántica.

Por ejemplo, ya en marzo de 2025, tuvieron una actualización de la web en la que eliminaron el 70Esta limitación impide que los datos lingüísticos (ejemplos gramaticales, reglas sintácticas, morfología) puedan ser procesados automáticamente por herramientas computacionales, buscadores especializados o sistemas de aprendizaje profundo. La motivación técnica de este trabajo radica, por tanto, en la superación de las barreras impuestas por el reconocimiento óptico de caracteres (OCR) convencional sobre este corpus específico de la ALMG, para generar corpus con morfologías clasificadas para entrenamiento de modelos. Las gramáticas lingüísticas de estas lenguas presentan una complejidad tipográfica elevada, que incluye tablas anidadas, glosas interlineales y estructuras jerárquicas que los sistemas tradicionales suelen interpretar de manera errónea. Ante este escenario, surge la oportunidad de emplear Modelos de Lenguaje de Gran Tamaño (LLMs), para comprender la arquitectura lógica de estos documentos institucionales y transformarla en datos estructurados bajo el estándar XML. Este proyecto busca proporcionar una solución eficiente que reduzca la carga de trabajo manual en la digitalización del patrimonio de la ALMG, garantizando que la información sea interoperable, consultable y apta para la generación de documentación científica profesional mediante LaTeX.

Documentos Publicados



Según la Ley de la Academia de las Lenguas Mayas de Guatemala, establece que dentro de sus funciones está traducir y publicar, previo cumplimiento de las leyes de la materia, códigos, leyes, reglamentos y otros textos legales o de cualquier naturaleza que se juzgue necesario a los idiomas Mayas.

Además de crear, implementar e incentivar programas de publicaciones bilingües y monolingües, para promover el conocimiento y uso de los idiomas mayas para fortalecer los valores culturales.

Para lo anterior la Academia ha realizado varias publicaciones en diferentes idiomas Mayas, y algunas de ellas se presentan a continuación:

Para mayor información pueden ingresar a la página de [contacto](#)

Comunidad Lingüística	Tipo de Documento	Nombre del Documento (Idioma Maya)	Nombre del Documento (Idioma Español)	Descarga
Achi	Texto	jik'ob'al Ub'ee Tz'ib' Maya'ch'a'teem Achi	Gramática _Normativa Idioma Maya Achi	
Akateka	Texto	Ik'ti'ey Skuyb'anil	Fábulas	
Akateka	Texto	Ik'ti'ey Skuyb'anil	Fábulas (2024)	
Akateka	Texto	xolilal Yelalil Ya'ley Ti'e Akateko	Gramática Descriptiva Akateka	
Akateka	Texto	Tz'ib' b'il q'ichmam Akateka	Literatura Maya Akateka	
Akateka	Texto	Sb'i K'ultajb'al yet Q'ichmam Akateko	Toponimias Maya Akateko	

Figura 1.: Web de la ALMG

Agradecimientos

*A mi familia por estar ahí en todo momento.
A mi pareja por ayudarme en todo lo que puede y más.
A mi madre por dedicar su vida a sus hijos.
A mi padre por animarme con mi carrera en la ciencia.
A mis compañeros del Santander por guiarme y enseñarme cómo desarrollar software.
Sin ellos no habría podido hacer este trabajo.*

Pues hemos nacido para colaborar, al igual que los pies, las manos, los párpados, las hileras de dientes, superiores e inferiores. Obrar, pues, como adversarios los unos de los otros es contrario a la naturaleza.

Marco Aurelio (Meditaciones)

Solo los que se quemaron con el fuego aprendieron a usarlo.

Andrés Pedreño Muñoz
Cofundador de Torre Juana OST

Índice general

Resumen	v
1. Introducción	1
1.1. Definición del problema	1
1.2. Alcance y limitaciones	3
1.2.1. Restricciones de propiedad intelectual y publicación de datos . . .	3
2. Estado de la cuestión	7
2.1. Modelos de lenguaje de gran tamaño (LLMs) aplicados a OCR	7
2.1.1. Evaluación de la integridad estructural en LLMs	8
2.1.2. Modelos multimodales y evaluación mediante OCRBench v2	8
2.2. APIs de LLMs	10
2.2.1. Ecosistema actual	10
2.2.2. Análisis de la Volatilidad y Restricciones de Acceso	10
2.2.3. Configuración de Modelos: Parámetro de Temperatura	11
2.3. Transfer Learning: Sinergias Lingüísticas y Extrapolación Global	12
2.3.1. Inferencia multilingüe y pesos compartidos	12
2.3.2. El valor de los datos estructurados (XML/LaTeX)	12
2.3.3. Democratización del conocimiento lingüístico	13
2.4. Alternativas en modelos locales y soberanía tecnológica	13
2.4.1. Mistral OCR y la eficiencia europea	13
2.4.2. DeepSeek-OCR-2: El avance de los VLMs	13
2.4.3. IBM granite-docling: miniaturización y precisión	14
2.4.4. Ventajas de la implementación local	14
2.5. Metodología de evaluación y métricas de rendimiento en IA	14
2.5.1. Métricas de error de texto (CER y WER)	14
2.5.2. Métricas de clasificación y precisión general	15
2.6. Lingüística computacional: análisis morfosintáctico y glosas	16
2.6.1. Análisis morfológico en lenguas aglutinantes	16
2.6.2. Estructura sintáctica y variabilidad	17
2.6.3. El desafío de las glosas interlineales	17
2.7. Contexto sociolingüístico: lenguas minorizadas y preservación digital . . .	18
2.7.1. El concepto de lengua minorizada	18
2.7.2. El papel de la Academia de Lenguas Mayas de Guatemala (ALMG) .	19
2.7.3. Urgencia de la preservación estructurada	19
2.8. Lenguajes de marcado y estándares de representación documental	19
2.8.1. XML y la representación jerárquica de la información	20

2.8.2.	Mecanismos de validación estructural (DTD y XSD)	20
2.8.3.	Composición de textos mediante L ^A T _E X	20
2.8.4.	Paradigmas de intercambio de datos: XML frente a JSON	21
2.8.5.	Paradigmas de almacenamiento lingüístico: XML estructurado frente a corpus paralelos convencionales	21
3.	Objetivos del trabajo	23
3.1.	Objetivo general	23
3.2.	Objetivos específicos	23
4.	Metodología y entorno de desarrollo	25
4.1.	Gestión del ciclo de vida del proyecto	25
4.1.1.	Planificación bajo restricciones	25
4.1.2.	Fases del desarrollo	25
4.1.3.	Control de versiones y documentación	26
4.2.	Configuración del entorno de desarrollo	26
4.2.1.	Gestión de entornos virtuales (Anaconda)	27
4.2.2.	Dependencias y librerías críticas	27
4.2.3.	Estructura de directorios	28
4.3.	Gestión de claves de API y seguridad de la información	28
4.3.1.	Externalización de credenciales	28
4.3.2.	Protección del repositorio y prevención de fugas	28
4.3.3.	Integridad de los datos de la ALMG	29
4.4.	Selección y descripción del corpus de trabajo	29
4.4.1.	Criterios de selección	29
4.4.2.	Caracterización del material documental	29
4.4.3.	Conformación del test set (Conjunto de pruebas)	30
5.	Diseño e implementación del sistema	31
5.1.	Arquitectura general del sistema	31
5.1.1.	Flujo de trabajo del pipeline	31
5.1.2.	Organización de los módulos del sistema	32
5.1.3.	Estructuración y Formalización del Esquema XML y DTD	33
5.1.4.	Definición Formal de la Gramática de Datos (DTD)	34
5.2.	Módulo de procesamiento de documentos (<code>pdf_processor.py</code>)	35
5.2.1.	Lógica de extracción y segmentación de bytes	35
5.2.2.	Gestión de la integridad y fidelidad del documento	35
5.2.3.	Selección de rangos y modularidad	36
5.3.	Motor de generación mediante IA (<code>ai_generator.py</code>)	36
5.3.1.	Configuración y parametrización del modelo	36
5.3.2.	Estrategia de inferencia: Few-Shot Learning	37
5.3.3.	Ingeniería de prompts estructural	37
5.3.4.	Limpieza de respuesta (Parsing)	37

5.4.	Estructuración, corrección y validación XML	37
5.4.1.	Definición de la gramática de datos (DTD)	38
5.4.2.	Corrección heurística automática (<code>xml_corrector.py</code>)	38
5.4.3.	Validación de integridad estructural (<code>xml_validator.py</code>)	38
5.5.	Motor de conversión y tipografía científica (<code>converter.py</code>)	39
5.5.1.	Maapeo semántico XML–LATEX	39
5.5.2.	Tratamiento de glosas y caracteres especiales	39
5.5.3.	Automatización de la compilación	40
5.6.	Automatización de experimentos y orquestación (<code>experiment_runner.py</code>)	40
5.6.1.	Diseño del barrido de hiperparámetros	41
5.6.2.	Flujo de orquestación modular	41
5.6.3.	Persistencia y versionado de resultados	41
6.	Experimentación y Análisis de Resultados	43
6.1.	Contextualización de resultados	43
6.2.	Diseño Experimental y Metodología de Evaluación	47
6.3.	Análisis Empírico del Parámetro de Temperatura	47
6.3.1.	Comportamiento Global y No Monotonidad del Sistema	48
6.3.2.	El “Punto Dulce” (<i>Sweet Spot</i>) frente al Determinismo Absoluto	48
6.3.3.	Resiliencia Estructural e Insensibilidad Térmica en Páginas Estables	49
6.3.4.	Correlación entre la Complejidad del Contenido y la Sensibilidad Térmica	50
6.4.	Evaluación Comparativa de Arquitecturas Multimodales (Modelos)	52
6.4.1.	Dominancia Sistémica de la Arquitectura Avanzada (<code>gemini-3.5-flash</code>)	52
6.4.2.	El Comportamiento Paradójico de los Modelos de Escala Reducida (<code>Lite</code>)	53
6.4.3.	Elasticidad de la Brecha Intermodelo en Función de la Complejidad del <i>Layout</i>	54
6.4.4.	No Linealidad Genérica del Rendimiento frente al Nomenclátor de Versión	55
7.	Conclusiones y trabajo futuro	57
7.1.	Síntesis del trabajo desarrollado	57
7.1.1.	Conclusiones del experimento con diferentes temperaturas	57
7.1.2.	Conclusiones del experimento con diferentes modelos	59
7.2.	Grado de cumplimiento de los objetivos	60
7.3.	Lecciones aprendidas y desafíos técnicos	61
7.3.1.	La fragilidad de la dependencia de APIs externas	61
7.3.2.	La IA como motor, la ingeniería como cinturón de seguridad	61
7.3.3.	Complejidad lingüística maya	62
7.4.	Líneas de investigación futura	62
7.4.1.	Migración a modelos locales y soberanía tecnológica	62
7.4.2.	Expansión del corpus lingüístico	62
7.4.3.	Desarrollo de una plataforma de consulta lingüística	63

7.4.4. Optimización del pipeline mediante RAG	63
7.4.5. Colaboración institucional y difusión de datos	63
A. Anexo I. Esquemas en xml	69
Lista de acrónimos	73

Índice de figuras

1.	Web de la ALMG	VIII
1.1.	Comparación OCR vs. Kaqchikel	2
2.1.	Curvas de precisión y cobertura en función del parámetro b del F1-Score .	16
2.2.	Ejemplo de glosa interlineal	18
6.1.	Glosa interlineal en el documento de entrada (pdf)	45
6.2.	Glosa interlineal en el documento de salida a partir del Latex compilado (pdf)	47

Índice de tablas

1.1. Comparativa de disponibilidad de modelos 2025-2026	4
2.1. Comparativa de proveedores de modelos de IA (Junio 2026)	10
5.1. Evolución de la nomenclatura XML durante el diseño del esquema estructural	33
6.1. Relación entre dificultad documental e impacto de la creatividad	51

Índice de Listados

5.1. Esquema XML refinado para la representación de glosas interlineales . . .	34
6.1. Esquema XML refinado para la representación de glosas interlineales . . .	45
6.2. Esquema Latex para la representación de glosas interlineales	46
A.1. Kaqchikel XML generado por el LLM a partir de la imagen de la página original	69
A.2. Document Type Definition	72
A.3. Ejemplo del esquema XML inicial utilizado para representar glosas interlineales	73

1. Introducción

1.1. Definición del problema

La digitalización de obras lingüísticas complejas se enfrenta a un obstáculo fundamental: la dicotomía entre la representación visual y la estructura lógica. El problema central se desglosa en los siguientes puntos críticos:

1. Naturaleza heterogénea de las fuentes (PDFs no procesados)

A diferencia de los documentos digitales nativos, el corpus de trabajo presenta una gran disparidad en su calidad técnica. Se ha identificado que un volumen significativo de los archivos PDF no son documentos con texto seleccionable y fondo limpio, sino fotografías directas de las páginas del libro físico. Estas fuentes carecen de cualquier pre-procesamiento de imagen o capa de texto subyacente, lo que implica: Ruido visual: Presencia de sombras, texturas del papel y falta de contraste entre la tipografía y el fondo. Inexistencia de metadatos de texto: La imposibilidad de extraer caracteres mediante scripts sencillos de lectura de PDF, obligando al sistema a realizar una interpretación visual completa de la imagen.

2. Insuficiencia del OCR convencional

Los sistemas de OCR tradicionales están diseñados para la extracción de texto lineal en documentos con una disposición de página sencilla como la parte izquierda de la figura 1.1. Sin embargo, la amalgama de una baja calidad de imagen junto con la arquitectura tipográfica y análisis sintácticos y morfológicos de las gramáticas como en la parte derecha de la figura 1.1, genera fallos sistemáticos en:

- Sistemas de numeración jerárquica:

Divisiones en secciones, tablas y disposiciones en vertical que suelen confundirse con el texto horizontal.

- Glosas y ejemplos multilingües:

La alternancia entre el castellano y la lengua maya, a menudo en bloques adyacentes, que el OCR estándar tiende a mezclar al no reconocer los límites visuales de las columnas o tablas.

- Estructuras morfológicas complejas:

Paradigmas verbales que, al ser extraídos de una "foto como texto plano, se convierten en caracteres inconexos.

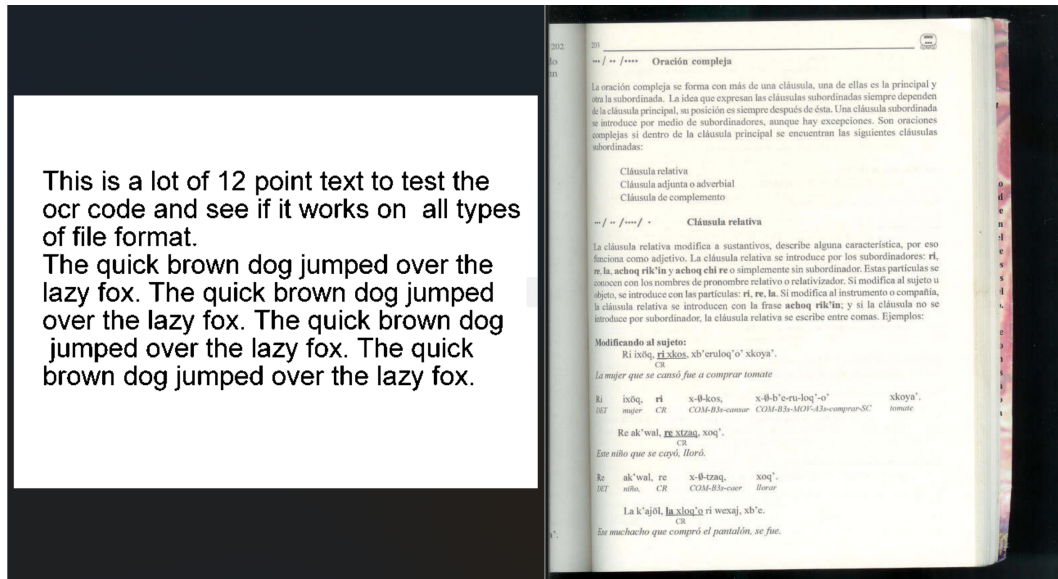


Figura 1.1.: Comparación OCR vs. Kaqchikel

- Estructuras sintácticas de subrayado:

Interpretaciones de segmentos de las oraciones subrayadas mediante análisis sintáctico.

- Texto sobrante:

a menudo al escanear un libro se recogen en la imágenes caracteres de las páginas adyacentes, manchas que dificultan la identificación de las letras, dibujos decorativos a inicio de página. . .

3. La pérdida de información al emplear PDF digitales

El formato PDF actúa aquí como un mero contenedor de imágenes. Al carecer de etiquetas semánticas (ej. `¡titulo!`, `¡ejemplo!`, `¡regla_gramatical!`), la información queda aislada de cualquier posibilidad de procesamiento masivo, análisis computacional o migración automática a formatos modernos. Se puede apreciar la diferencia entre ambos archivos de información comparando la figura (imagen) `pagina_original_Kaqchikel` en el código A.1.

4. El Desafío de la Inconsistencia Estructural en la IA

Incluso mediante el uso de LLMs multimodales, capaces de "ver" y procesar estas fotos de libros, existe una tendencia a la generación de errores sintácticos en el marcado XML (etiquetas mal cerradas o jerarquías invertidas), especialmente en páginas de alta densidad informativa como la páginas con análisis sintácticos o morfológicos. Parte de este problema se debe a la creatividad de los LLMs que

puede generar alucinaciones incluso empleando la metodología esquema de respuesta (response schema).

1.2. Alcance y limitaciones

El desarrollo de este Trabajo de Fin de Máster está condicionado por factores de carácter ético, legal y técnico que delimitan el marco de actuación. A continuación, se detallan las restricciones que han definido el alcance final de la investigación.

1.2.1. Restricciones de propiedad intelectual y publicación de datos

El corpus bibliográfico utilizado en este proyecto, consistente en gramáticas normativas de lenguas mayas, es propiedad intelectual de la Academia de Lenguas Mayas de Guatemala (ALMG). Como resultado de esta titularidad, se han establecido las siguientes limitaciones en cuanto a la difusión de datos:

- Prohibición de redistribución: No se permite la subida de los archivos PDF originales a repositorios públicos como GitHub. El acceso a las fuentes debe realizarse exclusivamente a través del portal institucional de la ALMG.
- Publicación de resultados parciales: El alcance de la publicación de resultados se limita a la estructura XML generada y a los fragmentos utilizados en el conjunto de pruebas (test set), preservando la integridad de la obra original.
- Lógica de replicabilidad: Para solventar esta limitación, el proyecto se enfoca en la liberación del código fuente (scripts de procesamiento), permitiendo que otros investigadores con acceso legítimo a los documentos puedan ejecutar el pipeline de digitalización de forma independiente.

Cabe destacar que el 70 % de los documentos mayas publicados en 2025 ya no están disponibles porque se han eliminado del catálogo de la web de la ALMG sin previo aviso. Esto provocó una gran dificultad en el trabajo ya que se tuvo que proseguir con los pocos documentos que se habían guardado en local antes de la actualización del catálogo.

Acotación de la carga de trabajo

De acuerdo con las directrices marcadas por la tutoría académica y con el fin de ajustar el proyecto a una carga de trabajo estrictamente acotada de 250 horas, se ha definido un alcance modular y selectivo:

- Selección de test set: En lugar de proceder a la digitalización de los libros completos, lo cual excedería el tiempo asignado en el cronograma, se ha optado por un análisis empírico profundo sobre una muestra representativa. Se han seleccionado un rango de 2 a 3 páginas de alta complejidad por cada idioma (Kaqchikel, K'iche', Mam y Q'anjob'al) para conformar el conjunto de validación final (Ground Truth).

- **Priorización técnica:** Se ha priorizado el perfeccionamiento de los módulos de validación estructural (`xml_corrector.py`) y la traducción automatizada a LaTeX sobre el procesamiento masivo de volumen, garantizando la robustez de la arquitectura antes de su escalabilidad futura.

Limitaciones técnicas de la API de Gemini

La dependencia de servicios de inteligencia artificial en la nube ha supuesto uno de los mayores desafíos logísticos del proyecto, debido a cambios imprevistos en las políticas de uso de Google Cloud y AI Studio, así como a la obsolescencia programada de sus arquitecturas:

- **Restricción drástica de cuotas de uso:** Durante el transcurso del desarrollo, el sistema sufrió una reducción crítica en los límites de peticiones permitidas. Se pasó de un margen operativo de 1.500 usos diarios (Requests Per Day - RPD) a un límite restrictivo de apenas 20 peticiones al día para los modelos en el nivel gratuito tal y como se ve en la tabla 1.1. Esta limitación del 98.6 % en la capacidad de cómputo obligó a reestructurar el flujo de trabajo, priorizando ejecuciones de prueba extremadamente precisas y limitando la posibilidad de realizar procesamientos masivos de páginas en una sola jornada.

Tabla 1.1.: Comparativa de disponibilidad de modelos 2025-2026

Modelos	Disponibilidad 2025 RPD	Disponibilidad 2026 RPD
gemini 1.5 flash	500	Deprecated
gemini 1.5 flash-lite	1500	Deprecated
gemini 2.0 flash	200	Deprecated
gemini 2.0 flash-lite	500	Deprecated
gemini 2.5 flash	20	20
gemini 2.5 flash-lite	20	20
gemma 3 27b	15.000	Deprecated
gemma 4 27b	No se había lanzado	Solo texto (al quitarle PDF)
gemini-3-flash-preview	No se había lanzado	20
gemini-3.1-flash-lite	No se había lanzado	500
gemini-3.5-flash	No se había lanzado	20

- **Reducción en la disponibilidad y ciclo de vida de los modelos:** Inicialmente, el entorno permitía el acceso concurrente a una variedad de entre 4 y 5 modelos de lenguaje distintos (incluyendo versiones experimentales y estables). No obstante, esta disponibilidad se vio mermada a solo 1 o 2 modelos activos por ventana de desarrollo debido a la depreciación masiva de arquitecturas estables. Específicamente, hitos de la plataforma como el del 29 de septiembre de 2025 supusieron la eliminación silenciosa de modelos de referencia como `gemini-1.5-pro` y `gemini-1.5-flash`, restringiendo la capacidad de realizar comparativas directas de rendimiento intramodelo de forma simultánea.

- Impacto en la metodología y presupuesto de tokens: Estas limitaciones técnicas externas impusieron una estricta economía de tokens y de tiempo. Se ha calculado matemáticamente el coste por llamada del pipeline mediante la ecuación de consumo estructural de tokens 1.3, considerando el prompt base y las capas de información visual y estructural que se envían a la API en cada iteración.

$$\text{Tokens por llamada} = 5,000 \text{ (prompt)} \quad (1.1)$$

$$+ [2,000 \text{ (imagen PDF)} + 2,000 \text{ (esquema XML)}] \times 2 \quad (1.2)$$

$$= 13,000 \text{ tokens} \quad (1.3)$$

Considerando las restricciones de la API para cuentas de desarrollo sin facturación asociadas a un techo de 250.000 tokens por minuto (Tokens Per Minute - TPM) y un límite de 10 solicitudes por minuto (RPM), la ventana de transmisión quedó restringida físicamente a un máximo de 19 llamadas simultáneas por minuto, forzando la inclusión de retrasos controlados en el script orquestador de 10 segundos en los reintentos. Sin embargo, tal y como se ha comentado anteriormente, la limitación más restrictiva fue la de peticiones por día (RPD - Request Per Day), ya que es más probable alcanzar 20 llamadas en un día que 19 llamadas en un minuto.

- Otra problemática de la API de gemini es que en momentos de alta demanda, esta no te devuelve resultados, te da error de servidor sobrecargado, lo que ha obligado a que en el proyecto se controle este flujo implementando un reintento a las 15 segundos.

Ante la inviabilidad operativa de las cuotas comerciales y los intentos fallidos de implementar arquitecturas locales basadas en la familia Gemma —debido a que gemma-3 fue completamente retirado por el proveedor y gemma-4 arrojó errores de servidor 500 al procesar de forma nativa la capa visual de los PDFs—, se optó estratégicamente por reorientar el pipeline hacia un modelo alternativo con límites específicos: gemini-3.1-flash-lite. Este cambio permitió eludir el bloqueo de cuota diaria (al ofrecer un techo de 500 peticiones), asumiendo una penalización marginal del 10% en la tasa de error de caracteres (CER) frente a los modelos de mayor escala, lo que obligó a robustecer los algoritmos heurísticos de post-procesamiento en el core del sistema.

2. Estado de la cuestión

2.1. Modelos de lenguaje de gran tamaño (LLMs) aplicados a OCR

Tradicionalmente, el Reconocimiento Óptico de Caracteres (OCR) se ha abordado mediante técnicas de visión artificial centradas en la detección de patrones de píxeles y la segmentación de glifos tal y como sostiene Singh et al. (2015). Herramientas clásicas como Tesseract o motores basados en redes neuronales convolucionales (CNN) han demostrado una alta eficacia en textos lineales y documentos digitales nativos. Sin embargo, su rendimiento decrece significativamente ante documentos con estructuras tipográficas complejas, ruido visual o disposiciones no lineales de la información. En los últimos años, la irrupción de los Modelos de Lenguaje de Gran Tamaño (LLMs) ha provocado un cambio de paradigma, evolucionando hacia lo que se conoce como OCR Estructurado o Document AI. A diferencia del OCR convencional, que se limita a transcribir caracteres, los LLMs con capacidades multimodales (Visual-Language Models o VLMs) procesan el documento como un todo semántico (Shen et al., 2025). Esta metodología se fundamenta en los siguientes pilares:

- Comprensión contextual y multimodalidad: Los modelos de última generación, como la familia Gemini empleada en este trabajo, integran codificadores visuales y de lenguaje. Esto permite al sistema no solo "ver" los caracteres, sino comprender la relación espacial entre ellos (Ding et al., 2026).

En el contexto de las gramáticas mayas, esta capacidad es crítica para diferenciar una glosa interlineal de un cuerpo de texto estándar, basándose en la disposición física en la página.

- Capacidad de inferencia y reconstrucción: A diferencia de los sistemas tradicionales que fallan ante caracteres ambiguos o manchas en el papel (comunes en las fotos de libros de la ALMG), los LLMs utilizan su conocimiento previo del lenguaje para inferir palabras probables, actuando simultáneamente como motor de extracción y corrector ortográfico contextual (Shen et al., 2025).
- Generación de salida estructurada: Uno de los mayores avances aplicados a este proyecto es la capacidad de estos modelos para seguir instrucciones complejas de formato (prompting). Mientras que un OCR tradicional devuelve texto plano (.txt), los LLMs pueden ser instruidos para encapsular la información directamente en lenguajes de marcado como XML. Esto elimina la necesidad de capas interme-

días de análisis de layout (Layout Analysis), permitiendo que el modelo genere la jerarquía de capítulos, secciones y subsecciones de forma nativa.

- Resiliencia ante fuentes heterogéneas: Se ha observado que los LLMs presentan una robustez superior al procesar documentos que carecen de metadatos de texto, como es el caso de las capturas fotográficas de libros antiguos. Su entrenamiento masivo en conjuntos de datos diversos les permite normalizar variaciones en la iluminación, curvatura de página y texturas del papel que suelen ser insalvables para algoritmos de binarización clásicos.

No obstante, la aplicación de LLMs al OCR introduce desafíos específicos, principalmente su naturaleza estocástica. Al ser modelos probabilísticos, existe el riesgo de *“alucinaciones estructurales”* (etiquetas XML inexistentes o mal cerradas). Esta limitación justifica la implementación de módulos de validación externa y corrección estructural, tales como el script `xml_corrector.py` desarrollado en este TFM, para garantizar que la potencia de comprensión del modelo se ajuste estrictamente a la gramática formal (DTD) requerida.

2.1.1. Evaluación de la integridad estructural en LLMs

Con la aparición de los Modelos de Lenguaje de Gran Tamaño (LLMs) capaces de generar código o lenguajes de marcado (XML, JSON), ha surgido la necesidad de evaluar la consistencia sintáctica. A diferencia del texto plano, una salida estructurada debe cumplir con reglas de jerarquía y anidamiento. La evaluación en este ámbito se realiza mediante:

- Validación de esquema: Comprobación booleana de que el documento generado es “bien formado” (well formed) y cumple con un estándar formal (como un DTD o XSD). Un DTD (por sus siglas en inglés, Document Type Definition o Definición de Tipo de Documento) es un conjunto de reglas que define la estructura, los elementos (etiquetas) y los atributos permitidos en un archivo XML o SGML. En el caso del proyecto se ha empleado el siguiente DTD A.2.
- Estabilidad estructural: Análisis de la tasa de éxito del modelo para cerrar etiquetas y mantener la profundidad del árbol jerárquico sin colapsar ante una alta densidad de información. Esta métrica es más genérica que la validación de esquema y por tanto más fácil de comprobar. Si la salida tiene estabilidad estructural se dice que es válida (valid).

2.1.2. Modelos multimodales y evaluación mediante OCRBench v2

La evolución reciente de la Inteligencia Artificial ha dado lugar a los denominados modelos multimodales o *Large Multimodal Models* (LMMs). Estos sistemas son capaces de procesar simultáneamente diferentes modalidades de información, como texto, imágenes, audio o vídeo, integrando todas ellas dentro de un mismo espacio de representación semántica.

En el contexto de la digitalización documental, los modelos multimodales han supuesto un cambio de paradigma respecto a los sistemas OCR tradicionales. Mientras que los motores OCR clásicos se limitan a reconocer caracteres de forma secuencial, los LMMs combinan reconocimiento visual, comprensión contextual y razonamiento lingüístico. Esto les permite interpretar estructuras complejas como tablas, diagramas, glosas interlineales o documentos con disposiciones tipográficas no convencionales.

Desde un punto de vista arquitectónico, estos modelos suelen estar compuestos por tres bloques principales:

- Un codificador visual (*Vision Encoder*), encargado de transformar la imagen en representaciones numéricas.
- Un mecanismo de proyección o alineamiento multimodal, que traduce las características visuales al espacio semántico utilizado por el modelo lingüístico.
- Un Modelo de Lenguaje de Gran Tamaño (LLM), responsable de interpretar la información visual y generar la salida textual correspondiente.

Durante el entrenamiento, estas arquitecturas son expuestas a grandes volúmenes de pares imagen-texto, aprendiendo a asociar elementos visuales con sus descripciones lingüísticas. Posteriormente suelen aplicarse fases de ajuste fino (*fine-tuning*) e instrucción (*instruction tuning*) orientadas a tareas específicas como OCR, extracción documental o razonamiento visual.

Para evaluar de forma objetiva las capacidades de estos sistemas, han surgido diversos benchmarks especializados. Entre ellos destaca OCRBench v2 Fu et al., 2025, considerado uno de los conjuntos de evaluación más completos para tareas de reconocimiento y comprensión documental.

OCRBench v2 incluye más de 10.000 pares pregunta-respuesta verificados manualmente y evalúa ocho capacidades fundamentales: reconocimiento de texto, localización, extracción de información, parsing estructural, cálculo, comprensión, referencia espacial y razonamiento. El benchmark incorpora además documentos complejos, contenido manuscrito, tablas y estructuras visuales que representan escenarios próximos a los encontrados en aplicaciones reales de digitalización.

Los resultados publicados muestran que incluso los modelos multimodales más avanzados continúan presentando dificultades en tareas de parsing documental y comprensión estructural. En la clasificación de 2026 destacan modelos como Gemini 3 Pro Preview, Gemini 2.5 Pro, NVIDIA Nemotron Omni y Qwen3-Omni, que obtienen resultados superiores al resto en tareas de extracción documental. Sin embargo, los autores señalan que la mayoría de arquitecturas todavía presentan limitaciones significativas en percepción de layouts complejos, parsing de elementos jerárquicos y razonamiento sobre estructuras documentales.

Esta observación resulta especialmente relevante para el presente Trabajo de Fin de Máster, ya que las gramáticas normativas de la ALMG contienen glosas interlineales, segmentaciones morfológicas y disposiciones tipográficas complejas que exigen capacidades de parsing estructural avanzadas. Por ello, la elección de modelos multimodales

modernos como Gemini constituye un requisito fundamental para alcanzar niveles de precisión adecuados en el proceso de digitalización.

2.2. APIs de LLMs

2.2.1. Ecosistema actual

En el ecosistema tecnológico de 2026, el acceso a Modelos de Lenguaje de Gran Tamaño se realiza mayoritariamente a través de interfaces de programación de aplicaciones (APIs) bajo un modelo de computación en la nube (Serverless AI). Este paradigma permite a los desarrolladores integrar capacidades cognitivas avanzadas sin la necesidad de gestionar infraestructura de hardware costosa. Sin embargo, el mercado se caracteriza por una alta volatilidad en los precios y, especialmente, en las políticas de cuotas de uso gratuito para investigación académica. En la tabla 2.2.1, se presenta una comparativa de las principales APIs disponibles, considerando las versiones de los modelos que lideran el sector en el presente año usando como fuente la web Cloudzero - LLM API Comparison In 2026.

Proveedor / Modelo	Plataforma	Req/día	Req/min	Tokens/min	Ventana	Max. Salida	Notas clave
gemini-2.5-flash	Google API	20	15	1M	1M	8K	Tier gratuito permanente.
gemini-3.1-flash-lite	Google API	1500	15	250K	1M	8K	Límites restrictivos en free tier.
gemini-2.0-flash	Google API	20	15	1M	1M	8K	Multimodal. Opción legacy.
llama-3.3-70b-versatile	GroqCloud	1,000	30	6,000	128K	32K	Velocidad 300-400 TPS.
llama-3.1-8b-instant	GroqCloud	14,400	30	30,000	128K	8K	El más permisivo de Groq.
llama-4-scout-17b	GroqCloud	1,000	30	30,000	128K	16K	Arquitectura nueva.
llama-3.3-70b	Cerebras Cloud	1M tok/día	30	60k-100k	8K	8K	2,600 TPS.
qwen3-32b	Cerebras Cloud	1M tok/día	30	60,000	64K	32K	Pool compartido.
mistral-small-3.1	La Plateforme	1B tok/mes	500	500,000	128K	8K	1B tokens/mes.
DeepSeek v3 / r1	DeepSeek Platf.	Sin límite	Dinám.	Dinám.	128K	8K	5M tokens registro.
gpt-4o / mini	OpenAI API	—	—	—	128K	16K	Requiere depósito.
claude-sonnet/haiku	Anthropic API	—	—	—	200K	8K	No tiene API free.

Tabla 2.1.: Comparativa de proveedores de modelos de IA (Junio 2026)

2.2.2. Análisis de la Volatilidad y Restricciones de Acceso

Uno de los hallazgos más relevantes durante el desarrollo de este TFM ha sido la inestabilidad de los servicios API como infraestructura para la digitalización lingüística. A diferencia de años anteriores, donde las cuotas para desarrolladores eran amplias, en 2026 se ha detectado una política de “ajuste agresivo” por parte de los grandes proveedores

(específicamente Google Cloud/AI Studio). Se ha documentado, mediante la bitácora de desarrollo de este trabajo, que las condiciones de acceso para el modelo Gemini sufrieron una degradación crítica sin previo aviso:

- Colapso de cuota: El margen operativo se redujo de 1.500 peticiones diarias a solo 20, lo que representa una caída del 98.6
- Restricción de modelos: La oferta de arquitecturas disponibles para pruebas concurrentes pasó de 8 modelos activos a únicamente 5, limitando la capacidad de realizar estudios comparativos de rendimiento intramodelo.

Esta situación subraya la importancia del objetivo específico 6 de este trabajo: la evaluación de la portabilidad hacia modelos locales. La dependencia de APIs comerciales, aunque ofrece una potencia de inferencia superior y capacidades multimodales nativas para procesar fotos de libros, introduce un riesgo de “apagón técnico” que puede comprometer cronogramas de investigación académica estricta. En conclusión, mientras que APIs como DeepSeek ofrecen el mejor ratio coste-beneficio en términos de tokens, Google sigue liderando en ventana de contexto, a pesar de la fragilidad de sus cuotas para el estamento de usuarios no corporativos.

2.2.3. Configuración de Modelos: Parámetro de Temperatura

Para optimizar el rendimiento de un LLM en tareas de digitalización estructurada, no solo es determinante la elección de la API, sino también el ajuste de sus hiperparámetros de generación y la selección de la arquitectura adecuada para la carga de trabajo.

La temperatura es un hiper parámetro que regula el grado de aleatoriedad o “creatividad” del modelo durante la fase de muestreo de tokens. Normalmente se define con un número en el rango del 0 al 1 (siendo 0 el mínimo y 1 el máximo grado de temperatura), aunque en Gemini se regula de 0 a 2. Técnicamente, actúa sobre la función softmax de la capa de salida del modelo, modificando la distribución de probabilidad de las palabras candidatas:

- Temperaturas bajas (T aprox 0.0 a 0.3): El modelo se comporta de forma cuasi-determinista, seleccionando siempre los tokens con mayor probabilidad estadística. En el contexto de este TFM, este rango es esencial para garantizar que la estructura XML sea coherente y que la transcripción de las gramáticas mayas sea fiel al original, minimizando la “alucinación” de etiquetas.
- Temperaturas altas ($T \geq 0.7$): El modelo aplanar la curva de probabilidad, otorgando oportunidades a tokens menos frecuentes. Aunque esto favorece la creatividad en redacción literaria, en la digitalización de documentos técnicos suele derivar en el colapso estructural del XML (etiquetas mal cerradas o duplicadas) y en la invención de términos lingüísticos inexistentes.

2.3. Transfer Learning: Sinergias Lingüísticas y Extrapolación Global

El aprendizaje por transferencia (Transfer Learning) representa uno de los pilares más avanzados de la Inteligencia Artificial moderna. Se define como la capacidad de un modelo de computación neuronal para aplicar el conocimiento adquirido en una tarea previa (fuente) a una tarea nueva pero relacionada (objetivo). En el marco de este TFM, esta técnica adquiere una dimensión crítica: el procesamiento de lenguas minorizadas con recursos computacionales limitados.

2.3.1. Inferencia multilingüe y pesos compartidos

Los Modelos de Lenguaje de Gran Tamaño (LLMs) han sido entrenados sobre corpus masivos que incluyen cientos de idiomas. Durante este proceso, las redes neuronales identifican estructuras universales de la lengua (como la relación entre sujetos, verbos y complementos). A través del algoritmo de retropropagación o backpropagation, el modelo ajusta sus pesos internos para minimizar el error de predicción. Conneau et al. (2020)

Cuando el sistema procesa una lengua de la familia maya, como el Kaqchikel, no parte de cero. Aprovecha la "memoria técnica" de estructuras gramaticales similares encontradas en otros idiomas. Este fenómeno permite que el aprendizaje de una lengua minoritaria .ayude.^a otras; por ejemplo, la comprensión de la morfología aglutinante o el uso de glosas en el K'iche' prepara los circuitos del modelo para ser mucho más eficiente al enfrentarse al Mam o, por extensión, a lenguas no relacionadas pero igualmente complejas (como el quechua o el ainu).

2.3.2. El valor de los datos estructurados (XML/LaTeX)

La mayor barrera para la digitalización de lenguas minoritarias es la .escasez de datos" (Data Scarcity). Sin embargo, el sistema desarrollado en este trabajo soluciona este problema mediante la generación de contenido de alta fidelidad. Aceleración del Fine-Tuning: Los archivos XML y LaTeX que genera la aplicación constituyen lo que en IA llamamos "Gold Standard.^o datos de referencia. Con una cantidad mínima de estas páginas estructuradas, es posible realizar un ajuste fino (fine-tuning) de modelos más pequeños y locales. Eficiencia en pocos ejemplos (Few-shot Learning): Como se observa en la arquitectura de carpetas de este proyecto (módulo data/few-shot), el modelo es capaz de aprender la jerarquía de una gramática completa tras ver solo dos o tres ejemplos bien etiquetados. Esta capacidad de generalización es extrapolable a cualquier lengua del mundo: si el pipeline es robusto para la complejidad de las lenguas mayas, su aplicación a otras familias lingüísticas requiere una inversión de tiempo y datos drásticamente inferior.

2.3.3. Democratización del conocimiento lingüístico

La metodología propuesta en este TFM establece un precedente para la preservación del patrimonio digital global. Al transformar "fotos de libros.^{en} "datos estructurados validados", estamos creando el combustible necesario para que los algoritmos de back-propagation de futuros modelos comprendan lenguas que, de otro modo, quedarían fuera del radar tecnológico. En conclusión, el Transfer Learning permite que el esfuerzo dedicado a digitalizar la Academia de Lenguas Mayas de Guatemala se convierta en un activo científico universal. La estructura lógica aprendida y los mecanismos de corrección implementados (`xml_corrector.py`) funcionan como un esquema transferible que garantiza que, con muy poca información inicial, cualquier lengua minorizada pueda alcanzar el mismo nivel de accesibilidad digital que las lenguas mayoritarias.

2.4. Alternativas en modelos locales y soberanía tecnológica

A pesar de la potencia de inferencia que ofrecen las APIs en la nube, el desarrollo de este TFM ha evidenciado la vulnerabilidad que supone la dependencia de servicios de terceros. Ante las restricciones de cuota y la volatilidad de los modelos comerciales, el estado del arte en 2026 apunta hacia la adopción de modelos de "pesos abiertos" (Open Weights) que pueden ser ejecutados en infraestructura local o entornos controlados como Google Colab. A continuación, se analizan las alternativas más competitivas para el OCR estructurado de gramáticas lingüísticas:

2.4.1. Mistral OCR y la eficiencia europea

El modelo Mistral OCR se ha consolidado como una de las herramientas más robustas para la comprensión de documentos complejos. Su arquitectura está optimizada para el análisis de layout, permitiendo identificar regiones de texto, tablas y fórmulas con una precisión cercana a los modelos propietarios. Su principal ventaja radica en su capacidad para procesar documentos multilingües (A Bhat et al., 2026), lo que lo convierte en un candidato ideal para actuar como motor secundario en la extracción de textos en Kaqchikel o K'iche'.

2.4.2. DeepSeek-OCR-2: El avance de los VLMs

La familia de modelos DeepSeek-OCR representa la vanguardia en los Modelos de Visión-Lenguaje (VLMs). A diferencia del OCR tradicional, DeepSeek-OCR-2 utiliza una arquitectura de extremo a extremo que traduce directamente la imagen del PDF en texto estructurado. Se ha demostrado especialmente eficaz en la preservación de jerarquías XML, reduciendo significativamente la tasa de etiquetas mal cerradas en comparación con modelos generales no especializados en visión (Wei et al., 2026).

2.4.3. IBM granite-docling: miniaturización y precisión

Uno de los hallazgos más relevantes para este trabajo es el modelo Granite-Docling-258M de IBM. Su reducido tamaño (258 millones de parámetros) permite su ejecución en entornos de hardware restringido, como instancias gratuitas de Colab o servidores con GPUs de gama media. Este modelo se complementa con la librería Docling, diseñada específicamente para la exportación de PDFs complejos a formatos estructurados (Markdown o XML). La implementación de Granite-Docling permitiría a la aplicación de este TFM procesar volúmenes masivos de páginas sin incurrir en costes de API ni depender de límites de peticiones por minuto.

2.4.4. Ventajas de la implementación local

La transición o convivencia con estos modelos locales aportaría al sistema tres beneficios estratégicos según Xu et al. (2024):

- Privacidad y ética: Garantiza que los documentos de la Academia de Lenguas Mayas de Guatemala no sean procesados por servidores externos de terceros, manteniendo la soberanía de los datos.
- Determinismo y control: Permite un control total sobre hiperparámetros como la temperatura y el top-p, sin las variaciones impredecibles que suelen introducir las actualizaciones silenciosas (silent updates) de las APIs comerciales.
- Escalabilidad sin coste: Elimina la "economía de tokens", permitiendo la digitalización de libros completos de la ALMG una vez validado el pipeline en el test set.

2.5. Metodología de evaluación y métricas de rendimiento en IA

La validación de sistemas basados en Inteligencia Artificial y Visión Artificial aplicados a la digitalización documental requiere un marco metodológico estructurado que permita cuantificar la precisión de los resultados de forma matemática y objetiva. En la literatura científica, la evaluación de modelos de extracción y estructuración de datos se divide generalmente en dos dimensiones ortogonales: la fidelidad de la transcripción textual y la exactitud de la clasificación y asignación jerárquica estructural.

2.5.1. Métricas de error de texto (CER y WER)

Para medir la calidad de la transcripción en tareas de Reconocimiento Óptico de Caracteres (OCR) y Procesamiento de Lenguaje Natural (PLN), el estándar de la industria se fundamenta en la cuantificación de la distancia de edición (Echavarria & Le Meur, 2025). Esta métrica computa el número mínimo de operaciones elementales necesarias para transformar una cadena de texto generada por el modelo en una cadena de texto

de referencia o Ground Truth. CER (Character Error Rate): Evalúa el error a nivel de carácter y se define matemáticamente como la suma de sustituciones (S), inserciones (I) y eliminaciones (D) necesarias para igualar las cadenas, normalizada mediante la división por el número total de caracteres de la cadena de referencia (N), tal como se expresa en la ecuación:

$$\text{CER} = (S + I + D) / N$$

No obstante, para el análisis de textos con alta densidad de caracteres especiales, resulta metodológicamente más preciso definir esta tasa a partir de la distancia InDel (Insertion-Deletion). Esta métrica constituye una variación de la distancia de Levenshtein clásica que restringe las operaciones permitidas exclusivamente a inserciones y eliminaciones. Funcionalmente, este comportamiento se implementa asignando un peso de 2 a cualquier operación de sustitución, asimilándola a un proceso secuencial de eliminación seguido de una inserción. El CER derivado es la métrica fundamental para determinar la robustez del modelo ante ruidos visuales, variaciones tipográficas o distorsiones en el soporte físico del documento.

WER (Word Error Rate): Mide la tasa de error a nivel de palabra. Siguiendo el mismo principio de distancia de edición, el WER cuantifica el volumen de palabras sustituidas, insertadas o eliminadas, normalizado estrictamente por la longitud total —medida en número de palabras— del texto de referencia. Mientras que el CER detecta fallos de lectura y distorsiones gráficas individuales, el WER actúa como un indicador macro de la capacidad del sistema para preservar la coherencia léxica y semántica del texto original.

2.5.2. Métricas de clasificación y precisión general

Más allá de la mera transcripción tipográfica, cuando un modelo multimodal debe clasificar y etiquetar información (como identificar títulos, glosas interlineales o estructuras jerárquicas), se emplean métricas analíticas derivadas de la matriz de confusión estadística:

- Exactitud (Accuracy): Se define como la proporción de predicciones correctas (tanto verdaderos positivos como verdaderos negativos) frente al total de casos evaluados. En el contexto de documentos estructurados, esta métrica permite cuantificar globalmente la correcta asignación de etiquetas semánticas a lo largo del corpus analizado.
- Precisión (Precision) y Cobertura (Recall): La precisión mide la capacidad del sistema para evitar la inclusión de falsos positivos, determinando la fiabilidad de las etiquetas asignadas. Por su parte, la cobertura (frecuentemente denominada exhaustividad en la literatura clásica) cuantifica la capacidad del modelo para localizar y recuperar la totalidad de los elementos positivos reales presentes en el documento. Para entender mejor la diferencia entre accuracy, precision y recall, ver la figura 2.1. En la digitalización de textos complejos, estas métricas permiten diagnosticar si el modelo está omitiendo secciones analíticas indispensables o si, por el contrario, está generando contenido espurio o alucinaciones estructurales.

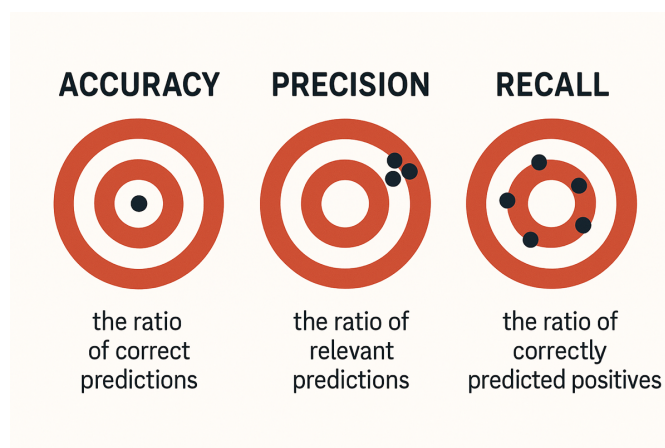


Figura 2.1.: Curvas de precisión y cobertura en función del parámetro β del F1-Score

- F1-Score y su generalización: Representa la media armónica entre la Precisión y la Cobertura, ofreciendo una métrica de rendimiento equilibrada, especialmente valiosa cuando los conjuntos de datos presentan un fuerte desbalance de clases. En escenarios de evaluación específicos donde se requiera priorizar la minimización de omisiones frente a la introducción de falsos positivos (o viceversa), se puede emplear la métrica generalizada F_β -score. Esta variante introduce un peso relativo mediante un parámetro β , formalizado de la siguiente manera:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precisión} \times \text{cobertura}}{(\beta^2 \times \text{precisión}) + \text{cobertura}}$$

Mediante la alteración del parámetro β , la comunidad científica puede ponderar el impacto de la cobertura sobre la precisión según las necesidades específicas del modelado lingüístico.

2.6. Lingüística computacional: análisis morfosintáctico y glosas

El procesamiento de gramáticas no se limita a la transcripción de caracteres; requiere una comprensión profunda de los componentes que estructuran el conocimiento lingüístico. A diferencia de las lenguas indoeuropeas, las familias mayas presentan desafíos específicos en su representación digital.

2.6.1. Análisis morfológico en lenguas aglutinantes

Las lenguas mayas se caracterizan por ser altamente sintéticas y aglutinantes. Esto significa que una sola palabra puede contener múltiples morfemas (raíces, prefijos y sufijos) que indican persona, tiempo, aspecto, modo y otras categorías gramaticales. En la digitalización, es crítico que el sistema no rompa estas cadenas de caracteres de forma

arbitraria. El análisis morfológico extraído por la IA debe preservar la integridad de los afijos, ya que la pérdida de un solo fonema puede alterar completamente el sentido de una regla gramatical descrita en el libro original.

2.6.2. Estructura sintáctica y variabilidad

La sintaxis de estas lenguas suele seguir órdenes de palabras menos comunes en idiomas mayoritarios, como VSO (Verbo-Sujeto-Objeto) o VOS. A continuación exponemos algunos ejemplos de diferentes ordenes:

1. Estructura VSO (Verbo-Sujeto-Objeto):
2. Ejemplo (Irlandés): Ólann (V) Seán (S) uisce (O).
3. (Traducción: Juan bebe agua).
1. Estructura VOS (Verbo-Objeto-Sujeto):
2. Ejemplo (Malgache): Manasa (V) lamba (O) Rabe (S).
3. (Traducción: Rabe lava la ropa).
1. Estructura VOS (Verbo-Objeto-Sujeto):
2. Ejemplo: Kuxlan (V) kaxlan wa (O) ri achi (S).
3. (Traducción: El hombre come pan).
4. Análisis: Kuxlan (come) es el verbo, kaxlan wa (pan) es el objeto, y ri achi (el hombre) es el sujeto.

Las gramáticas de la ALMG explican estas variaciones mediante ejemplos complejos. El sistema de digitalización estructurada debe ser capaz de identificar cuándo una oración es un ejemplo de "sintaxis normativa" cuándo es una "excepción dialectal", encapsulando cada caso en su etiqueta XML correspondiente para permitir búsquedas especializadas por parte de lingüistas.

2.6.3. El desafío de las glosas interlineales

El componente más crítico y difícil de procesar en estas obras son las glosas interlineales. Este es un estándar académico de representación que suele constar de tres o cuatro niveles de información alineados verticalmente:

1. Línea de texto original: La oración en lengua maya.
2. Línea de traducción: El significado global en castellano.
3. Línea de segmentación morfológica: Desglose de la palabra en sus componentes.

4. Línea de etiquetas gramaticales: Descripción técnica de cada morfema (ej. 1Sg, DET, PL).

Para ver un ejemplo de esta estructura, se recomienda consultar la figura 2.2, que ilustra claramente cómo cada nivel de información se corresponde verticalmente, lo que representa un desafío significativo para los sistemas de OCR tradicionales.

Xkilöq' jun uqaj ri ka'i' achi'a'.					
SVT					
Los dos hombres compraron un corte.					
X-Ø-ki-löq'	jun	uqaj	ri	ka'i'	achi'-a'.
COM-B3s-A3p-comprar	IND	corte	DET	dos	hombre-PL

Figura 2.2.: Ejemplo de glosa interlineal

El OCR convencional falla estrepitosamente ante esta estructura, ya que tiende a leer de forma horizontal, mezclando el maya con las etiquetas técnicas y el castellano, generando un resultado carente de sentido. El uso de LLMs multimodales en este trabajo permite realizar una alineación semántica vertical. El modelo “entiende” que la palabra de la línea 1 se corresponde con la etiqueta de la línea 3, permitiendo que nuestra aplicación genere una estructura XML donde cada componente lingüístico está perfectamente vinculado. Esta es la base para que, en el futuro, estos datos puedan alimentar motores de traducción automática o diccionarios inteligentes.

2.7. Contexto sociolingüístico: lenguas minorizadas y preservación digital

La diversidad lingüística del mundo se encuentra en una situación de vulnerabilidad sin precedentes. Según datos de la UNESCO, se estima que cada dos semanas desaparece una lengua, llevando consigo un sistema único de conocimiento y cosmovisión. En este escenario, la tecnología actúa como un arma de doble filo: puede acelerar la asimilación hacia lenguas mayoritarias o, si se emplea correctamente, servir como un motor de revitalización y resiliencia cultural.

2.7.1. El concepto de lengua minorizada

El término “lengua minorizada” hace referencia a aquellos idiomas que, por procesos históricos y políticos, han sido desplazados de los espacios públicos, administrativos y, fundamentalmente, digitales. En el caso de Guatemala, el país alberga una extraordinaria riqueza lingüística con 22 idiomas mayas, además del xinka y el garífuna. A pesar de contar con millones de hablantes, lenguas como el Kaqchikel, el K'iche' y el Mam sufren una brecha digital lingüística. La falta de herramientas básicas en el ecosistema digital (teclados predictivos, correctores, traductores o bases de datos consultables) condena

a estas lenguas a una existencia analógica que dificulta su transmisión a las nuevas generaciones de “nativos digitales”.

2.7.2. El papel de la Academia de Lenguas Mayas de Guatemala (ALMG)

La ALMG, como entidad rectora de este patrimonio, ha realizado una labor histórica de codificación a través de sus gramáticas normativas. Estos documentos son la “Constitución” lingüística de cada idioma. Sin embargo, su almacenamiento mayoritario en archivos PDF estáticos (o fotos de libros, como se ha analizado en este trabajo) limita su impacto. La digitalización estructural que propone este TFM no es solo un cambio de formato; es un acto de soberanía tecnológica. Al convertir estas gramáticas en datos procesables, estamos permitiendo que las lenguas mayas entren en la era de la Inteligencia Artificial en igualdad de condiciones semánticas.

2.7.3. Urgencia de la preservación estructurada

La preservación digital no debe confundirse con el simple escaneo de documentos. Un libro escaneado es una “tumba digital” si la información no es recuperable por una máquina. La aportación de este trabajo radica en la creación de un corpus estructurado. Disponer de la normativa gramatical en formatos como XML y $\text{\LaTeX}$ permite:

- Crear aplicaciones educativas interactivas.
- Desarrollar sistemas de búsqueda avanzada para investigadores y hablantes.
- Alimentar modelos de lenguaje locales (revisados en el apartado 2.4) que no dependan de la infraestructura de grandes corporaciones tecnológicas.

En conclusión, la digitalización de las lenguas Kaqchikel, K’iche’ y Mam mediante técnicas de IA avanzada es una respuesta necesaria ante el riesgo de “extinción digital”. Este TFM se alinea con los objetivos internacionales de desarrollo sostenible, promoviendo el acceso a la información y la protección de la diversidad cultural a través de la innovación tecnológica.

2.8. Lenguajes de marcado y estándares de representación documental

La transformación de información no estructurada, como los documentos en formato PDF o imágenes, en conocimiento digital procesable requiere el uso de estándares que garanticen la interoperabilidad, la validación y la calidad estética. En el ámbito académico y editorial, el ecosistema tecnológico actual se apoya en lenguajes de marcado semántico y sistemas de composición tipográfica avanzada.

2.8.1. XML y la representación jerárquica de la información

El lenguaje XML (Extensible Markup Language) se mantiene como el estándar de facto para el almacenamiento y el intercambio de documentos complejos. A diferencia de lenguajes como el HTML, cuyo propósito es la visualización, el XML permite la definición de etiquetas personalizadas que describen la naturaleza semántica de los datos.

En el campo de las Humanidades Digitales y la Lingüística, el XML es fundamental por las siguientes razones:

- **Estructuración semántica:** Permite encapsular la jerarquía de un libro (partes, capítulos, secciones) y elementos específicos (glosas, morfemas, ejemplos) en una estructura de árbol legible tanto por humanos como por máquinas.
- **Preservación a largo plazo:** Al ser un formato basado en texto plano y no propietario, garantiza que la información digitalizada sea recuperable independientemente de la evolución del software.
- **Separación de contenido y formato:** Facilita que un mismo conjunto de datos sea exportado a múltiples formatos (PDF, ePub, Web) manteniendo la integridad del contenido.

2.8.2. Mecanismos de validación estructural (DTD y XSD)

La fiabilidad de los datos estructurados depende de su cumplimiento con una gramática formal. En el estado del arte actual, el DTD (Document Type Definition) y el XSD (XML Schema Definition) son las tecnologías que actúan como esquemas de validación.

Estos componentes permiten establecer un “contrato estructural” que define:

- La obligatoriedad y el orden de aparición de los elementos.
- La jerarquía de anidamiento permitida.
- Las reglas gramaticales que la salida de un sistema automático (como un motor de IA) debe respetar.

Esta validación es crítica en flujos de trabajo automatizados, ya que permite detectar de forma inmediata errores de formación o inconsistencias lógicas antes de que los datos sean integrados en bases de datos o compilados para su publicación.

2.8.3. Composición de textos mediante $\text{\LaTeX}$

Para la generación de documentación científica de alta calidad, $\text{\LaTeX}$ representa el estándar internacional en las comunidades académicas de ingeniería y ciencias exactas. Se define como un sistema de composición de textos que separa la escritura del diseño editorial.

Sus principales aportaciones al estado de la técnica incluyen:

- **Excelencia tipográfica:** Gestión superior de la microtipografía, el espaciado y la renderización de caracteres especiales, algo esencial para la representación de glifos lingüísticos y alfabetos fonéticos.
- **Automatización de estructuras:** Capacidad para generar de forma dinámica índices, referencias cruzadas y numeraciones complejas a partir de fuentes de datos estructuradas.
- **Tratamiento de glosas lingüísticas:** Mediante paquetes especializados (como `expex` o `gb4e`), $\text{\LaTeX}$ permite la alineación vertical de múltiples niveles de análisis lingüístico, garantizando una presentación profesional que los procesadores de texto convencionales no pueden alcanzar de forma automatizada.

2.8.4. Paradigmas de intercambio de datos: XML frente a JSON

En la actualidad, existe un debate técnico entre el uso de JSON (JavaScript Object Notation) y XML. Mientras que JSON ha ganado terreno en el desarrollo de aplicaciones web y APIs ligeras debido a su baja verbosidad y facilidad de lectura en entornos de programación modernos, el XML sigue siendo superior en el ámbito de la documentación técnica y lingüística.

La superioridad del XML en este contexto radica en su capacidad nativa para gestionar documentos extensos con jerarquías profundas y en la madurez de sus herramientas de validación estructural (como XPath y DTD). En tareas de digitalización masiva mediante Modelos de Lenguaje de Gran Tamaño (LLMs), el XML proporciona un marco de contención más rígido que ayuda a mitigar la naturaleza estocástica de la generación de la IA, asegurando que la estructura del documento original se preserve fielmente.

2.8.5. Paradigmas de almacenamiento lingüístico: XML estructurado frente a corpus paralelos convencionales

En el ámbito de la lingüística computacional aplicada a lenguas minorizadas, la elección del modelo de datos determina críticamente la riqueza del conocimiento analizable por una máquina. Históricamente, los enfoques tradicionales de traducción automática se han apoyado en los denominados *corpus paralelos* (*parallel corpora*), los cuales se limitan a alinear oraciones equivalentes en dos o más idiomas (generalmente bajo una estructura lineal de texto plano o bitexto). Sin embargo, para la preservación y el estudio filológico de gramáticas densas como las de la familia maya, este paradigma plano presenta deficiencias metodológicas insalvables frente al marcado jerárquico XML.

El inconveniente cardinal de los corpus paralelos radica en la pérdida irreversible de la información estructural y espacial de las glosas interlineales. Al reducir un texto normativo complejo a meros pares tipográficos aislados (oración en lengua maya frente a su traducción en castellano), se destruyen las capas intermedias esenciales de segmentación morfológica y categorización gramatical. Este achatamiento de la información reproduce un efecto análogo al fallo de lectura horizontal que adolecen los sistemas de OCR

convencionales, donde las columnas y niveles analíticos verticales se mezclan de forma caótica.

Frente a la planitud del corpus paralelo, la encapsulación del texto mediante etiquetas semánticas en XML proporciona un entorno de aprendizaje drásticamente superior para los Modelos de Lenguaje de Gran Tamaño (LLMs). Una Inteligencia Artificial comprende y procesa con mayor precisión la correlación entre una raíz morfológica aglutinante y su respectivo morfema técnico si estas variables se encuentran delimitadas en nodos jerárquicos explícitos (ej. `<glosa>`, `<segmentacion>`, `<categoria>`).

Esta delimitación por bloques sintácticos previene que la naturaleza estocástica de los modelos confunda la traducción libre con las reglas sintácticas subyacentes. En consecuencia, para capturar la gramática profunda y la semántica de una lengua minorizada sin sacrificar recursos científicos tan valiosos como las glosas normativas de la ALMG, la adopción de una arquitectura basada en XML estructurado se consolida en el estado de la técnica actual como una estrategia muy superior a la de un corpus paralelo convencional, garantizando que el conocimiento digitalizado sea verdaderamente explotable para la investigación lingüística avanzada.

3. Objetivos del trabajo

El propósito fundamental de este proyecto es la creación de un ecosistema tecnológico que permita la transición de gramáticas de lenguas mayas desde formatos estáticos y no estructurados hacia documentos digitales estructurados.

3.1. Objetivo general

Diseñar e implementar un sistema automatizado basado en llamadas a APIs de Modelos de Lenguaje de Gran Tamaño (LLMs) para la extracción, estructuración y digitalización de gramáticas normativas de lenguas mayas minorizadas (Kaqchikel, K'iche, Mam y Q'anjob'al), transformando fuentes documentales complejas (incluyendo imágenes y PDFs sin capa de texto) en datos estructurados bajo el estándar XML y compilables en $\text{\LaTeX}$.

3.2. Objetivos específicos

Para dar cumplimiento al objetivo general, se han definido las siguientes metas técnicas y de investigación:

- **Desarrollar un pipeline modular de procesamiento:** Codificar una arquitectura en Python que integre la gestión de documentos PDF, el preprocesamiento de imágenes y la interacción con la API de Gemini, permitiendo un flujo de trabajo escalable y reproducible para el aumento de recursos de cualquier idioma.
- **Definir una gramática formal de datos:** Establecer un Documento de Definición de Tipo (DTD) que actúe como esquema de validación, garantizando que el modelo de IA genere etiquetas XML jerárquicamente coherentes con la estructura de un texto lingüístico (títulos, líneas, glosas, análisis sintáctico...).
- **Implementar herramientas de corrección estructural:** Programar módulos de post-procesamiento capaces de detectar y subsanar automáticamente inconsistencias sintácticas en el marcado XML, tales como etiquetas mal cerradas o jerarquías invertidas, propias de la generación estocástica de los LLMs.
- **Analizar empíricamente la influencia de la temperatura:** Realizar un estudio sistemático sobre el parámetro de temperatura del modelo para identificar el “punto dulce” que equilibre la precisión del OCR con la flexibilidad necesaria para interpretar estructuras tabulares y glosas complejas.

- **Automatizar la generación de documentación científica:** Desarrollar un motor de conversión que traslade los datos validados en XML a código $\text{\LaTeX}$, asegurando que la salida final mantenga los estándares de calidad tipográfica requeridos en el ámbito académico.
- **Manejo y control de errores en la aplicación:** Manejar los errores más comunes (ya bien sea por librerías o APIs externas) y generar cuadernos de bitácora o *logs* con los registros completos para entender y monitorear el flujo y los posibles errores durante la ejecución del programa.
- **Evaluar la portabilidad a modelos locales:** Investigar la viabilidad técnica de sustituir o complementar la API de Gemini con modelos de código abierto o locales (como Mistral OCR o DeepSeek-OCR), con el fin de explorar alternativas que reduzcan la dependencia de servicios de terceros y costes de API.

4. Metodología y entorno de desarrollo

Este capítulo detalla el marco metodológico, las herramientas técnicas y los procedimientos de seguridad empleados para la consecución de los objetivos. Se describe un enfoque centrado en la reproducibilidad y la eficiencia bajo restricciones de recursos.

4.1. Gestión del ciclo de vida del proyecto

El desarrollo de este Trabajo de Fin de Máster se ha articulado bajo una metodología iterativa e incremental, permitiendo ajustar el sistema a medida que se descubrían las particularidades de las gramáticas mayas y las limitaciones de la API de Gemini.

4.1.1. Planificación bajo restricciones

Siguiendo las directrices de la tutoría académica, el proyecto se diseñó para ser ejecutado en una carga de trabajo acotada de 250 horas. Esta restricción temporal ha sido un factor determinante en la toma de decisiones estratégicas, priorizando:

- La robustez del *pipeline* modular sobre el volumen de procesamiento masivo.
- El desarrollo de un *Test Set* representativo (de 2 a 3 páginas críticas por idioma) que permitiera validar el sistema de forma empírica sin necesidad de procesar los libros completos en esta fase inicial, optimizando el consumo de cuotas diarias de la API.

4.1.2. Fases del desarrollo

El ciclo de vida del software se ha dividido en cuatro fases principales, reflejadas en la bitácora de desarrollo y el repositorio:

- **Fase de exploración y prototipado (Zero-Shot):** En las primeras etapas, se realizaron pruebas de extracción de texto plano para evaluar la capacidad de visión de Gemini. Se detectó la necesidad de transicionar hacia una salida estructurada debido a la pérdida de jerarquía semántica.
- **Fase de ingeniería de prompts y estructuración (Few-Shot):** Se implementó la lógica de *Few-Shot Learning* (visible en la carpeta `data/few-shot`), proporcionando al modelo ejemplos de XML validado para guiar la generación. Metodológicamente, se optó por la técnica avanzada de pre-carga de historial (*History Pre-loading*) a nivel de SDK, inyectando los pares de imagen y XML directamente

en el búfer de la sesión antes del envío del mensaje del usuario, lo que optimizó drásticamente la latencia y la eficiencia en la transferencia de red. Durante esta fase se documentó el fenómeno de *overfitting* o pérdida de generalización cognitiva al incrustar capturas fijas directamente en el prompt; el modelo tendía a reproducir de forma literal metadatos espurios del ejemplo (como la cadena “Adidas” o las oraciones exactas de la muestra) en las nuevas páginas, lo que obligó a refinar la abstracción del prompt textual. En esta fase se definió el DTD inicial.

- **Fase de refinamiento estructural y validación:** Desarrollo de los módulos de post-procesamiento (`xml_corrector.py` y `xml_validator.py`). Se iteró sobre el código para subsanar los errores estructurales típicos del modelo (etiquetas mal cerradas o duplicadas). En esta fase se diseñó la lógica de escape del bucle de validación (*Validation Loop*): si tras el quinto intento de autoreparación iterativa el XML generado persiste en incumplir las especificaciones gramaticales del archivo DTD, el orquestador interrumpe las llamadas para evitar el bloqueo del *pipeline*, guardando la última traza con errores heurísticos y continuando con el flujo secuencial de ejecución.
- **Fase de experimentación y análisis:** Realización de pruebas sistemáticas variando la temperatura del modelo (de 0.0 a 1.5) para identificar el “punto dulce” de precisión, tal como se detalla en el análisis de resultados del Capítulo 5. Durante esta etapa (específicamente en los ensayos del 10 de febrero de 2026), se descubrió una limitación física insalvable en el procesamiento por bloques: el sistema se encuentra acotado a un máximo de 2 páginas simultáneas. Debido a que cada sílaba analizada lingüísticamente en las glosas exige un mínimo de 4 etiquetas XML anidadas (aproximadamente 6 líneas de marcado), un volumen superior satura la ventana máxima de tokens de salida de la API, provocando truncamientos estructurales fatales.

4.1.3. Control de versiones y documentación

La trazabilidad del proyecto se ha garantizado mediante el uso de un repositorio en GitHub, organizando el código en una arquitectura modular que separa la lógica de procesamiento (`core/`), la configuración (`config/`), la validación (`validator/`) y la evaluación (`evaluator/`). Este ordenamiento facilita la mantenibilidad del sistema y su posible escalabilidad futura.

4.2. Configuración del entorno de desarrollo

La estabilidad de un sistema de Inteligencia Artificial depende críticamente de la gestión de dependencias y la insolación del entorno de ejecución. Para este proyecto, se ha diseñado un ecosistema basado en estándares de la industria que garantiza la portabilidad del código.

4.2.1. Gestión de entornos virtuales (Anaconda)

Se ha utilizado Anaconda como gestor de entornos y paquetes, permitiendo aislar las librerías del TFM del resto del sistema operativo. Esta decisión técnica evita conflictos entre versiones de Python y facilita la replicabilidad por parte de otros investigadores.

- **Nombre del entorno:** `gemini-env`.
- **Versión de Python:** 3.12 (seleccionada por su optimización en la gestión de hilos y compatibilidad con las últimas librerías de Google AI).
- **Procedimiento de inicialización:** El entorno se activa mediante el comando `conda activate gemini-env`, asegurando que todas las variables de ruta apuntan a las binarias correctas.

Nota de resolución de entorno: Durante la fase inicial de despliegue, se documentó un conflicto de resolución de entorno derivado del uso del comando abreviado `py` en sistemas Windows. Dicho comando puenteaba el entorno virtual activo de Anaconda ejecutando el intérprete global del sistema, lo que impedía la localización de la SDK `google-gemai`. El problema se solventó forzando metodológicamente la sintaxis explícita `python main.py`, garantizando así la vinculación estricta de las variables de entorno de `gemini-env`.

4.2.2. Dependencias y librerías críticas

El sistema se apoya en un conjunto de librerías especializadas instaladas a través de `pip` y gestionadas mediante el archivo `requirements.txt`. Las dependencias clave se agrupan en tres categorías:

Interacción con modelos de IA

- **google-generativeai:** Librería principal para la comunicación con la API de Gemini (modelos 1.5 Pro, Flash y 2.5 Flash-lite). Gestiona el envío de imágenes y la recepción de respuestas estructuradas.

Procesamiento de documentos y visión

- **pdf2image:** Utilizada para convertir las páginas del PDF en imágenes de alta resolución, paso necesario para que el modelo multimodal pueda “ver” la disposición tipográfica de las gramáticas.
- **Pillow (PIL):** Empleada para el preprocesamiento de imágenes antes de su envío a la API.

Gestión y validación de datos

- **lxml:** Librería fundamental para el procesamiento de XML en Python. Se utiliza en el módulo `xml_validator.py` para contrastar la salida de la IA contra el archivo `gramatica.dtd`, asegurando que el documento cumpla con las reglas gramaticales definidas.
- **python-dotenv:** Para la carga segura de variables de entorno, evitando la exposición de claves de API en el código fuente.

4.2.3. Estructura de directorios

El entorno se ha organizado siguiendo una arquitectura limpia (*Clean Architecture*), donde el script principal `main.py` actúa como orquestador, invocando módulos específicos para cada tarea. Esta modularidad permite, por ejemplo, actualizar el motor de conversión a $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ (`converter.py`) sin afectar a la lógica de extracción de la IA (`ai_generator.py`).

4.3. Gestión de claves de API y seguridad de la información

En proyectos que dependen de servicios en la nube facturables o sujetos a cuotas restrictivas, la seguridad de las credenciales de acceso es una prioridad crítica. Durante el desarrollo de este TFM, se han implementado protocolos para asegurar que la API Key de Gemini sea gestionada de forma externa al código fuente.

4.3.1. Externalización de credenciales

Siguiendo los estándares de seguridad de software, se ha evitado la práctica de “hard-codear” (insertar directamente) las claves en los scripts de Python. En su lugar, se ha optado por un sistema de variables de entorno:

- **Inyección en tiempo de ejecución:** La clave se carga en la memoria del sistema operativo antes de ejecutar la aplicación mediante el comando `set GEMINI_API_KEY=[CLAVE.SECRETA]`.
- **Consumo mediante código:** El módulo `config/properties.py` utiliza la librería `os` para recuperar esta clave de forma dinámica, asegurando que el valor real nunca quede registrado en el historial de archivos del proyecto.

4.3.2. Protección del repositorio y prevención de fugas

Para garantizar que la información sensible no sea publicada accidentalmente en el repositorio de GitHub, se han aplicado las siguientes medidas:

- **Uso de .gitignore:** Se ha configurado un archivo de exclusión que impide que archivos de configuración local (como `.env`), registros de logs con trazas de depuración o carpetas de entorno virtual sean subidos al servidor público.

- **Aislamiento de logs:** El sistema de registro implementado en `core/logger_config.py` ha sido configurado para monitorizar el flujo de la aplicación sin volcar en los archivos de texto (`logs/*.log`) el contenido íntegro de las peticiones de API que contienen la clave de acceso.

4.3.3. Integridad de los datos de la ALMG

En cumplimiento con las restricciones de propiedad intelectual mencionadas en el Capítulo 1, el entorno de desarrollo se ha configurado para que los archivos PDF originales de la Academia de Lenguas Mayas de Guatemala residan únicamente en el almacenamiento local del investigador. El repositorio en la nube solo contiene los scripts de procesamiento y los resultados validados, garantizando el respeto a los derechos de autor de las fuentes originales.

4.4. Selección y descripción del corpus de trabajo

La validez de este estudio depende de la representatividad y complejidad del material documental seleccionado. Para ello, se ha conformado un corpus de trabajo compuesto por cuatro gramáticas de lenguas mayas, editadas por la Academia de Lenguas Mayas de Guatemala (ALMG), que representan diferentes variantes y desafíos técnicos: Kaqchikel, K'iche', Mam y Q'anjob'al.

4.4.1. Criterios de selección

Se han seleccionado estas obras basándose en tres criterios estratégicos:

- **Relevancia demográfica y representatividad documental:** El K'iche', el Kaqchikel, el Mam y el Q'anjob'al permiten evaluar el sistema sobre lenguas mayas con distinta disponibilidad documental, variedad tipográfica y complejidad estructural, lo que refuerza la representatividad del corpus utilizado.
- **Diversidad estructural:** Cada gramática presenta una disposición tipográfica distinta, lo que permite evaluar la resiliencia del sistema ante diferentes estilos editoriales.
- **Complejidad del soporte:** El corpus incluye tanto documentos digitales con capa de texto como archivos consistentes en fotografías de libros físicos, permitiendo testear las capacidades multimodales del modelo Gemini.

4.4.2. Caracterización del material documental

A continuación, se detallan las fuentes que integran el corpus de estudio:

- **Gramática Normativa Kaqchikel:** Representa el mayor desafío estructural del proyecto. Se ha identificado como una obra con alta densidad de glosas interlineales

y tablas de paradigmas verbales complejos. Especialmente, la página 172 de este documento se ha utilizado como “patrón de estrés” para las pruebas de temperatura debido a su saturación informativa.

- **Gramática Normativa K’iche’:** Este documento destaca por su rigor en el análisis morfológico. Ha servido para validar la capacidad de la IA en la extracción de segmentaciones lingüísticas y su correcta traducción al castellano en bloques adyacentes.
- **Gramática Normativa Mam:** Incorporada para asegurar que el pipeline sea capaz de manejar variaciones en el conjunto de caracteres Unicode y estructuras de numeración jerárquica que difieren levemente de las otras variantes.
- **Gramática Descriptiva Q’anjob’al:** Incluida para ampliar la validación del sistema a una cuarta lengua maya y comprobar su comportamiento ante una obra con características documentales y tipográficas distintas. Sus páginas 44, 94 y 201 forman parte del conjunto experimental empleado en la evaluación.

4.4.3. Conformación del test set (Conjunto de pruebas)

Dada la restricción de tiempo y potencia de cómputo (analizada en el Capítulo 1), no se ha procedido al procesado lineal de los libros completos. En su lugar, se ha construido un *Test Set* compuesto por una selección de 2 a 3 páginas de alta complejidad por cada idioma.

Este subconjunto de datos ha sido etiquetado manualmente para generar el *Ground Truth* necesario, permitiendo que el módulo `evaluator` realice comparativas estadísticas precisas sobre la estabilidad del sistema bajo diferentes configuraciones de temperatura.

5. Diseño e implementación del sistema

En este capítulo se describe la arquitectura técnica del sistema desarrollado, detallando la lógica de cada módulo y los mecanismos de integración entre el procesamiento de documentos, la inteligencia artificial y la generación de documentación científica.

5.1. Arquitectura general del sistema

El sistema se ha diseñado bajo una arquitectura modular y desacoplada, organizada en un *pipeline* de procesamiento secuencial con capas de validación. Esta estructura permite que cada componente cumpla una función única, facilitando el mantenimiento y la posible sustitución de módulos (como el cambio de modelo de IA) sin afectar al resto del ecosistema.

5.1.1. Flujo de trabajo del pipeline

El proceso de transformación de una gramática analógica en un documento estructurado sigue un flujo de cinco etapas:

1. **Capa de ingesta y preprocesamiento:** El sistema localiza los archivos PDF en el directorio `data/` y los transforma en imágenes de alta resolución compatibles con la entrada multimodal de la API.
2. **Capa de inferencia (Motor de IA):** Se envían las imágenes junto con un *prompt* especializado y ejemplos de entrenamiento (*few-shot*) para obtener una respuesta en formato XML.
3. **Capa de post-procesamiento y corrección:** La salida de la IA se somete a una limpieza automática para corregir errores sintácticos comunes en la generación estocástica (etiquetas mal formadas o duplicadas).
4. **Capa de validación estructural:** Se contrasta el XML generado contra el archivo de definición `gramatica.dtd` para asegurar el cumplimiento de la jerarquía lingüística.
5. **Capa de exportación y tipografía:** Una vez validado, el contenido se traslada a formatos finales: XML para almacenamiento de datos, $\text{\LaTeX}$ para publicación académica y PDF para mayor visibilidad.

5.1.2. Organización de los módulos del sistema

La implementación se refleja en la siguiente organización de directorios en el repositorio:

- **core/**: Contiene la lógica de negocio principal, incluyendo la interacción con la IA, la gestión de archivos y el corrector de XML.
- **config/**: Centraliza los parámetros del sistema, rutas de directorios y la gestión de la arquitectura técnica.
- **validator/**: Aloja las reglas formales (DTD) y el script de validación que garantiza la calidad de los datos.
- **latex_generator/**: Módulo especializado en la conversión de etiquetas XML a comandos de tipografía científica.
- **evaluator/**: Infraestructura para la comparación de resultados y cálculo de métricas de precisión frente al *test set*.

Esta arquitectura garantiza la trazabilidad absoluta de los datos: desde el PDF original hasta el archivo `.tex` final, cada paso es registrado y puede ser auditado mediante el sistema de *logs* centralizado.

Además de las carpetas anteriores, también existen directorios destinados al almacenamiento de datos y recursos auxiliares:

- **data/**: Contiene los datos de entrada al programa, como gramáticas en PDF, incluyendo los ejemplos de entrada y salida utilizados en el proceso de *few-shot learning*.
- **logs/**: Almacena de manera permanente todos los archivos de texto con registros de la aplicación para poder consultar los procesos que se han ejecutado en cada momento.
- **output/**: Carpeta por defecto para la salida del sistema y donde se almacenan las ejecuciones anteriores para futuros análisis y comparativas.
- **prompts/**: Aloja los archivos de texto con las diferentes versiones de los *prompts* que se han ido utilizando a lo largo del desarrollo.
- **test_set/**: Centraliza los conjuntos de pruebas empleados para la evaluación de la salida del programa.

5.1.3. Estructuración y Formalización del Esquema XML y DTD

La consecución de un corpus digitalizado con valor científico para la lingüística computacional exige que la salida generada por el Modelo de Lenguaje de Gran Tamaño (LLM) no solo sea precisa a nivel grafémico, sino que se ajuste estrictamente a una gramática de datos formal. Para ello, se ha diseñado un esquema jerárquico personalizado bajo el estándar XML, cuya validez sintáctica y estructural queda gobernada por una Definición de Tipo de Documento (DTD).

En las etapas iniciales de prototipado del sistema, se propuso una nomenclatura e infraestructura de marcado en castellano orientada a emular la disposición física y bidimensional de las páginas. El marcado inicial respondía al patrón heurístico mostrado en la Tabla 5.1.

Tabla 5.1.: Evolución de la nomenclatura XML durante el diseño del esquema estructural

Anterior	Nuevo	Descripción
glosa_interlineal	interlinear_gloss	Conjunto semántico a analizar
corpus_paralelo	parallel_phrase	Frases en idioma original y traducido
frase_original	original	Frase en idioma original (maya)
frase_español	translation	Frase en idioma traducido (español)
glosa	morpheme_analysis	Análisis morfológico
columna1	unit	Unidad a analizar
fila1	form	Parte de la frase original
fila2	gloss	Análisis morfológico del segmento de la frase original

La estructura inicial del marcado XML puede observarse en el Código A.3. Este diseño perseguía representar de forma explícita la disposición de las glosas interlineales mediante elementos que simulaban columnas y filas, facilitando la interpretación visual de la información lingüística.

Sin embargo, a través de un proceso de refinamiento iterativo, se descartó esta aproximación debido a dos deficiencias de diseño:

- Acoplamiento visual: las etiquetas como `<columna1>` o `<fila1>` forzaban al modelo de IA a realizar un análisis geométrico del soporte físico en lugar de una interpretación lingüística, lo que aumentaba la tasa de error por la distorsión espacial de las fuentes escaneadas.
- Incompatibilidad internacional: se requería una transición hacia los estándares anglosajones dominantes en el Procesamiento del Lenguaje Natural (PLN).

En consecuencia, el esquema se reestructuró por completo hacia una ontología abstracta en inglés, abstrayendo el diseño plano y encapsulando las unidades lingüísticas en árboles semánticos de análisis morféxico y sintáctico. Asimismo, se optimizó la flexibilidad de la estructura eliminando contenedores intermedios rígidos para secciones complejas, lo que dotó al *pipeline* de una mayor resiliencia ante la naturaleza estocástica del LLM. El esquema anterior se transforma en la estructura mostrada en el código 5.1.

Listado 5.1: Esquema XML refinado para la representación de glosas interlineales

```

1 <interlinear_gloss>
2   <parallel_phrase>
3     <original>k'am chi-∅ w-il-a'</original>
4     <translation>No lo veo</translation>
5   </parallel_phrase>
6
7   <morpheme_analysis>
8     <unit>
9       <form>k'am</form>
10      <gloss>NEG</gloss>
11    </unit>
12
13    <unit>
14      <form>chi-∅</form>
15      <gloss>INC-</gloss>
16    </unit>
17
18    ...
19  </morpheme_analysis>
20 </interlinear_gloss>

```

5.1.4. Definición Formal de la Gramática de Datos (DTD)

El contrato estructural definitivo que rige la validación sintáctica del sistema se encuentra formalizado en el archivo `gramatica.dtd`, cuyo código se detalla en el Código A.2.

Con este archivo DTD, el documento XML resultante se articula como un árbol jerárquico de herencia y adyacencia donde se diferencian dos grandes tipologías de elementos:

- Elementos de estructura editorial

(`document`, `page`, `heading`, `title`, `line`): `document` opera como el nodo raíz del sistema. Este contiene de forma iterativa uno o más elementos `page`. Cada página está dotada de un atributo obligatorio (`number`) para preservar la trazabilidad bibliográfica respecto al PDF original. El contenido interno de `page` es de tipo mixto y libre de contenedores rígidos; permite la aparición libre e intercalada de líneas ordinarias (`line`), títulos capitulares (`title`) y encabezados (`heading`), mitigando los bloqueos en el analizador sintáctico (*parser*) cuando la IA procesa estructuras transicionales de maquetación.

- Elementos de análisis lingüístico avanzado

(`interlinear_gloss`, `parallel_phrase`, `morpheme_analysis`, `syntactic_analysis`, `unit`): el elemento central de la digitalización semántica es `interlinear_gloss`. Como se especifica en la lógica relacional de la DTD, este nodo actúa como un orquestador que puede integrar, de manera opcional y secuencial, un bloque de

traducción directa (`parallel_phrase`), una capa de desagregación morfológica (`morpheme_analysis`) y una capa de segmentación de sintagmas y funciones gramaticales (`syntactic_analysis`). Las capas morfológica y sintáctica se componen obligatoriamente de una o más estructuras constitutivas denominadas `unit`. A nivel atómico, cada `unit` asocia unívocamente una forma morfológica o sintáctica concreta de la lengua maya (`form`) con su glosa explicativa parametrizada en castellano (`gloss`).

Mediante esta arquitectura relacional, el sistema no solo salvaguarda el texto plano de las gramáticas normativas, sino que genera un mapa posicional y gramatical explícito altamente estructurado, idóneo para el entrenamiento automatizado de futuros modelos cognitivos.

5.2. Módulo de procesamiento de documentos (`pdf_processor.py`)

El primer eslabón del *pipeline* es el módulo encargado de la ingesta y manipulación de los archivos fuente. Dado que las gramáticas de la ALMG presentan una naturaleza heterogénea, se ha implementado una estrategia de segmentación y extracción de flujo binario que permite aislar fragmentos del documento original sin alterar su estructura interna.

5.2.1. Lógica de extracción y segmentación de bytes

El script `pdf_processor.py` actúa como un gestor de *streams* de datos que opera directamente sobre la estructura del formato PDF. En lugar de realizar una rasterización de páginas, su función principal es segmentar el documento para extraer rangos específicos de páginas y servirlos en formato binario (*bytes*) de forma eficiente.

- Herramientas técnicas: se utiliza la librería PyPDF2, que permite realizar operaciones de lectura y escritura a bajo nivel sobre archivos PDF.
- Procedimiento: el módulo abre el archivo fuente mediante `PdfReader`, calcula el índice de las páginas solicitado realizando el ajuste de base 1 a base 0 requerido por la librería, y emplea `PdfWriter` para encapsular únicamente las páginas seleccionadas. El resultado no es una imagen, sino un objeto serializado en memoria (`io.BytesIO`) que contiene el PDF resultante, el cual se retorna como un flujo de *bytes*. Este enfoque asegura que la IA reciba la información más pura posible del documento, conservando su estructura vectorial nativa.

5.2.2. Gestión de la integridad y fidelidad del documento

Un factor crítico en este desarrollo es la preservación de la integridad del documento original durante el filtrado. Al trabajar con la manipulación de *bytes* en lugar de conversiones a imagen, el sistema garantiza:

- Ausencia de degradación: al evitar procesos de binarización o compresión de imágenes, el texto y los elementos gráficos del PDF mantienen su definición nativa, facilitando que el modelo multimodal interprete caracteres pequeños y estructuras complejas con mayor precisión.
- Eficacia en el entorno de producción: la entrega del contenido en formato de flujo binario permite una integración directa con los *endpoints* de la API de Gemini que aceptan archivos PDF como fuente, aprovechando la capacidad del modelo para “leer” la estructura interna del documento sin necesidad de procesar metadatos innecesarios o ruido visual generado por conversiones de formato.

5.2.3. Selección de rangos y modularidad

Para optimizar el consumo de *tokens* y asegurar el cumplimiento de las cuotas diarias de la API, el procesador incluye una lógica de extracción selectiva basada en rangos. Esta funcionalidad ha sido determinante para trabajar sobre el *Test Set* de forma aislada, permitiendo ejecutar pruebas rápidas, precisas y fácilmente reproducibles sin cargar volúmenes masivos de datos irrelevantes.

En resumen, `pdf_processor.py` garantiza que el sistema gestione la ingesta de documentos de manera modular y técnica, proporcionando a la Inteligencia Artificial una entrada de datos fiel al original y optimizada para el *pipeline* de digitalización estructural.

5.3. Motor de generación mediante IA (`ai_generator.py`)

Este módulo constituye el núcleo cognitivo del sistema. Su función es orquestar la comunicación con los Modelos de Lenguaje de Gran Tamaño (LLMs) de la familia Gemini, transformando las imágenes procesadas en datos estructurados mediante técnicas avanzadas de ingeniería de *prompts*.

5.3.1. Configuración y parametrización del modelo

El script `ai_generator.py` gestiona la forma en la que se realiza la llamada a la API del modelo generativo. La configuración de la petición se centra en tres parámetros fundamentales:

- Temperatura: parámetro central de la experimentación de este TFM, que regula el equilibrio entre determinismo y creatividad. Se ha implementado de forma variable para permitir el estudio sistemático de su impacto en la precisión estructural.
- Modelo: identifica el modelo utilizado para generar la respuesta. Esta constante se encuentra definida en el archivo `properties.py`.
- Contenido: engloba tanto el *prompt* principal como los ejemplos de *few-shot learning*, incluyendo sus respectivas entradas, salidas y explicaciones.

5.3.2. Estrategia de inferencia: Few-Shot Learning

Para garantizar que el modelo genere una estructura XML coherente con la gramática formal definida por la DTD, el sistema no utiliza una estrategia de *Zero-Shot Learning*, sino una aproximación basada en *Few-Shot Learning*.

A través del módulo `prompt_reader.py`, el sistema incorpora a la petición ejemplos reales formados por pares de entrada y salida compuestos por una imagen y su correspondiente XML validado, almacenados en la carpeta `data/few-shot`.

Esta técnica proporciona al modelo un patrón visual y sintáctico previo, reduciendo significativamente las alucinaciones estructurales y favoreciendo una aplicación consistente de las etiquetas XML definidas en el esquema de datos.

5.3.3. Ingeniería de prompts estructural

El *prompt* enviado al modelo ha sido diseñado para actuar como una guía técnica de generación. Entre las directrices incorporadas destacan las siguientes:

- Fidelidad textual: obligatoriedad de transcribir caracteres especiales, signos diacríticos y grafías propias de las lenguas mayas sin aplicar normalizaciones innecesarias.
- Jerarquía XML: instrucciones explícitas para encapsular la información siguiendo la estructura formal descrita en la DTD.
- Manejo de errores visuales: directrices sobre cómo proceder ante manchas de tinta, texto parcialmente oculto o imágenes borrosas, instando al modelo a emplear inferencia contextual para reconstruir palabras o segmentos incompletos.

5.3.4. Limpieza de respuesta (Parsing)

Dado que los LLMs suelen devolver sus respuestas encapsuladas en bloques de código Markdown mediante delimitadores de triple comilla invertida, `ai_generator.py` incorpora una fase de limpieza o *parsing* destinada a extraer exclusivamente el contenido XML generado.

Esta operación constituye un paso previo imprescindible para que la salida pueda ser procesada correctamente por los módulos de corrección y validación estructural, evitando que elementos de presentación ajenos al documento interfieran con el análisis sintáctico posterior.

5.4. Estructuración, corrección y validación XML

Uno de los mayores desafíos de emplear LLMs para la generación de datos es su naturaleza estocástica, que puede derivar en errores sintácticos de marcado. Para mitigar este riesgo, el sistema incorpora una capa de garantía de calidad (*Quality Assurance*, QA) que combina corrección heurística y validación formal.

5.4.1. Definición de la gramática de datos (DTD)

La estructura jerárquica de las gramáticas mayas se ha formalizado en el archivo `validator/gramatica.dtd`. Este documento actúa como el contrato estructural que debe cumplir cualquier salida generada por el sistema.

El DTD establece las relaciones jerárquicas entre los distintos elementos del documento, definiendo qué etiquetas pueden aparecer, en qué orden deben hacerlo y cuáles son sus restricciones estructurales. Gracias a este mecanismo, se garantiza que los documentos XML producidos por la Inteligencia Artificial mantengan una organización coherente y compatible con las fases posteriores de procesamiento.

5.4.2. Corrección heurística automática (`xml_corrector.py`)

Antes de someter la respuesta de la IA a una validación formal, el sistema invoca el módulo `xml_corrector.py`. Este componente aplica una serie de algoritmos de limpieza destinados a subsanar los errores más frecuentes detectados durante la fase de prototipado.

Entre las operaciones realizadas destacan:

- Cierre de etiquetas: detección y clausura automática de etiquetas que el modelo ha dejado abiertas debido a límites de generación o errores de inferencia.
- Eliminación de redundancias: supresión de cabeceras XML duplicadas, bloques explicativos ajenos al documento o fragmentos de texto generados fuera de la estructura esperada.
- Normalización estructural: verificación de la existencia de un único elemento raíz, corrigiendo situaciones en las que el modelo produce múltiples nodos principales incompatibles con la sintaxis XML.

Estas operaciones permiten aumentar significativamente la tasa de éxito de las validaciones posteriores sin alterar el contenido semántico generado por el modelo.

5.4.3. Validación de integridad estructural (`xml_validator.py`)

Una vez corregido, el documento XML es procesado por el módulo de validación. Utilizando la librería `lxml`, el sistema compara la estructura generada con las reglas definidas en el archivo DTD.

El resultado de este proceso es binario:

- Si el documento cumple todas las restricciones gramaticales, se autoriza su almacenamiento y posterior conversión a $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$.
- Si se detecta cualquier incumplimiento estructural, el error es capturado y registrado automáticamente en el sistema de *logs*.

Esta capacidad de detección ha permitido cuantificar experimentalmente la estabilidad de las diferentes configuraciones de temperatura analizadas en el Capítulo 5. De este modo, una ejecución puede clasificarse como fallida incluso cuando el contenido textual es correcto, siempre que la estructura jerárquica no respete las reglas definidas por la gramática formal.

La incorporación de esta capa de validación transforma una herramienta de generación textual en un sistema de extracción de datos con garantías académicas, asegurando que la información digitalizada sea estructuralmente consistente y apta para su integración en repositorios lingüísticos y bases de datos especializadas.

5.5. Motor de conversión y tipografía científica (`converter.py`)

La etapa final del sistema tiene como objetivo transformar los datos estructurados en XML en un documento de alta calidad tipográfica apto para su consulta, análisis y publicación académica. Para ello, se ha desarrollado el módulo `converter.py`, encargado de realizar la traducción semántica entre el modelo de datos XML y el lenguaje de composición tipográfica $\text{\LaTeX}$.

5.5.1. Mapeo semántico XML– $\text{\LaTeX}$

El convertidor implementa una lógica de transformación que asocia cada etiqueta XML con su representación correspondiente en $\text{\LaTeX}$, preservando la jerarquía definida en la gramática original.

Entre las transformaciones más relevantes se encuentran:

- Estructura jerárquica: los elementos que representan títulos, encabezados y divisiones documentales se convierten dinámicamente en comandos como `\chapter{}`, `\section{}` o `\subsection{}` según su nivel de profundidad.
- Cuerpo de texto: los bloques de contenido textual se normalizan para asegurar una correcta gestión de párrafos, saltos de línea y separación entre elementos, siguiendo los estándares de edición científica.

Este mecanismo garantiza que la organización lógica del documento permanezca inalterada durante todo el proceso de conversión.

5.5.2. Tratamiento de glosas y caracteres especiales

Uno de los principales retos técnicos abordados por este módulo es la correcta representación de los caracteres específicos de las lenguas mayas, incluyendo saltillos, vocales modificadas y otros símbolos lingüísticos presentes en las gramáticas normativas.

Para ello, el sistema incorpora las siguientes estrategias:

- Codificación Unicode: toda la salida se genera en formato UTF-8, permitiendo que motores de composición modernos como XeLaTeX o LuaLaTeX representen correctamente los caracteres especiales.
- Formateo de glosas: los ejemplos lingüísticos se estructuran mediante tablas, listas y entornos especializados que preservan la alineación entre formas originales, segmentaciones morfológicas y traducciones, evitando los problemas de desorganización visual característicos de las conversiones automáticas desde texto plano.

Estas medidas resultan fundamentales para garantizar la fidelidad lingüística del documento final.

5.5.3. Automatización de la compilación

El proceso de conversión se encuentra completamente automatizado. Una vez que el archivo XML supera satisfactoriamente la validación estructural, `converter.py` genera de forma inmediata un archivo `.tex` completo y listo para compilar.

Entre las tareas realizadas automáticamente destacan:

- Generación del preámbulo técnico necesario para la configuración del documento.
- Inclusión de los paquetes requeridos para el soporte de idiomas, codificación y maquetación.
- Construcción automática de la estructura jerárquica utilizada posteriormente por L^AT_EX para generar índices y referencias cruzadas.
- Producción de un archivo final preparado para su compilación directa a PDF sin necesidad de intervención manual.

Esta capacidad de automatización completa el ciclo de digitalización propuesto en este trabajo: la información que originalmente se encuentra en fotografías o documentos PDF se transforma en un recurso digital estructurado, consultable, reutilizable y presentado con una calidad tipográfica equivalente a la de una edición académica profesional.

5.6. Automatización de experimentos y orquestación (`experiment_runner.py`)

Para llevar a cabo un estudio empírico sobre el comportamiento de los LLMs en la digitalización lingüística, se ha desarrollado un módulo de orquestación denominado `experiment_runner.py`. Este componente permite ejecutar pruebas masivas de forma sistemática, eliminando el sesgo asociado al procesamiento manual y garantizando la reproducibilidad de los experimentos.

5.6.1. Diseño del barrido de hiperparámetros

El módulo está diseñado para realizar un barrido (*sweep*) sobre el parámetro de temperatura del modelo. Para ello, el orquestador ejecuta el *pipeline* completo de forma iterativa, variando la temperatura en un conjunto predefinido de valores, como por ejemplo 0.0, 0.1, 0.25, 0.5, 1.0 y 1.5.

Con el objetivo de reducir el impacto de la variabilidad inherente a los modelos generativos, el sistema permite configurar múltiples repeticiones para cada valor de temperatura. Habitualmente se emplean entre 10 y 15 ejecuciones por configuración experimental.

Esta estrategia permite calcular métricas agregadas y detectar comportamientos anómalos, especialmente en configuraciones de alta temperatura donde pueden aparecer fenómenos de inestabilidad estructural o pérdida de coherencia en la generación del XML.

5.6.2. Flujo de orquestación modular

El módulo `experiment_runner.py` actúa como coordinador del sistema, invocando secuencialmente los distintos componentes del *pipeline* para cada experimento.

El flujo de ejecución sigue el siguiente orden:

1. Obtención del documento de prueba mediante `pdf_processor.py`.
2. Ejecución de `ai_generator.py` utilizando la temperatura correspondiente a la iteración experimental.
3. Aplicación de las reglas de corrección implementadas en `xml_corrector.py`.
4. Validación formal del resultado mediante `xml_validator.py`.
5. Conversión del XML validado a L^AT_EX utilizando `converter.py`.

Esta organización modular permite reutilizar los mismos componentes tanto en experimentos automatizados como en ejecuciones individuales, garantizando la consistencia metodológica de todo el proceso.

5.6.3. Persistencia y versionado de resultados

Una de las características más relevantes de este módulo es su sistema de almacenamiento jerárquico de resultados. Cada ejecución se organiza automáticamente según criterios como el modelo empleado, el idioma procesado y la temperatura utilizada.

El sistema incorpora además un mecanismo de versionado incremental. Cada resultado generado se almacena con un identificador de versión secuencial (`v1`, `v2`, `v3`, etc.), permitiendo estudiar la estabilidad del modelo a lo largo de múltiples ejecuciones.

Gracias a este enfoque resulta posible analizar el grado de determinismo del sistema. Si varias ejecuciones producen exactamente la misma salida, puede inferirse un comportamiento altamente estable. Por el contrario, si aparecen diferencias significativas entre versiones generadas bajo la misma configuración, estas variaciones pueden cuantificarse y analizarse estadísticamente.

Adicionalmente, el orquestador registra información relativa al tiempo de respuesta de la API y monitoriza posibles excepciones durante la ejecución. Entre ellas se incluyen errores de conectividad, interrupciones del servicio o agotamiento de cuotas de uso. La captura y almacenamiento de estos eventos permite reanudar experimentos interrumpidos sin comprometer la integridad de los datos obtenidos.

Gracias a este nivel de automatización, el sistema trasciende el ámbito de una prueba conceptual y se convierte en una infraestructura experimental capaz de generar evidencia empírica sobre la fiabilidad de los Modelos de Lenguaje de Gran Tamaño en tareas de preservación y digitalización del patrimonio lingüístico. Los datos producidos por este módulo constituyen la base cuantitativa empleada posteriormente en el análisis de resultados presentado en el Capítulo 5.

6. Experimentación y Análisis de Resultados

Este capítulo aborda la evaluación empírica y sistemática del sistema automatizado de digitalización lingüística desarrollado en este trabajo. A través de una metodología basada en la ejecución secuencial y controlada de un corpus de prueba representativo (*Test Set*), compuesto por nueve páginas críticas pertenecientes a las gramáticas normativas de la Academia de Lenguas Mayas de Guatemala (ALMG), se analiza el comportamiento del *pipeline* bajo diferentes configuraciones operativas. Los resultados se presentan normalizados en términos de porcentaje de similitud estricta, métrica ortogonal e inversa al error a nivel de carácter (*Character Error Rate*, CER).

6.1. Contextualización de resultados

Antes de adentrarnos en el análisis detallado de los resultados, es importante contextualizar el flujo completo de la aplicación y comprender cómo se transforma una fuente documental original en un recurso digital estructurado.

Tal y como se ha descrito en capítulos anteriores, el sistema recibe como entrada un documento PDF procedente de las gramáticas normativas de la Academia de Lenguas Mayas de Guatemala (ALMG) y genera automáticamente tres tipos de salida complementarios: un archivo XML estructurado, un documento LaTeX y un PDF compilado a partir de dicho código LaTeX.

Para ilustrar este proceso, a continuación se presenta una misma glosa interlineal extraída de la página 172 de la Gramática Normativa Kaqchikel. Se muestran tanto la fuente original en formato PDF (6.1) como las diferentes representaciones generadas por el sistema: el archivo XML (6.1), el código LaTeX (6.2) y el PDF compilado resultante (6.2).

Si se compara la imagen original con el PDF compilado, se aprecia una diferencia significativa en la calidad visual. Mientras que la fuente original corresponde a una fotografía o digitalización de una página impresa, la salida final constituye un documento generado digitalmente mediante composición tipográfica avanzada. Sin embargo, el contenido lingüístico permanece inalterado, demostrando la capacidad del sistema para preservar con precisión tanto la información textual como la estructura gramatical presente en la fuente original.

Este proceso de digitalización estructurada resulta especialmente relevante en el contexto de las lenguas minorizadas, ya que transforma documentos estáticos en recursos digitales reutilizables, buscables y procesables por sistemas informáticos. De este modo,

las gramáticas dejan de ser únicamente documentos de consulta humana para convertirse también en recursos explotables por herramientas de lingüística computacional, sistemas de búsqueda avanzada y futuros modelos de inteligencia artificial.

Por otro lado, el código LaTeX generado permite observar cómo la información lingüística extraída del XML se transforma en una representación formal siguiendo los estándares internacionales utilizados en documentación lingüística. El sistema emplea el paquete **expex**, ampliamente utilizado en lingüística descriptiva y tipología lingüística para la representación de glosas interlineales.

A continuación se describen los principales elementos presentes en la salida LaTeX:

- `\par\medskip`: introduce una separación vertical entre bloques de contenido. Su función es mejorar la legibilidad del documento y delimitar visualmente cada ejemplo lingüístico.
- `\ex`: inicia un nuevo ejemplo lingüístico numerado. Este comando es proporcionado por el paquete **expex** y permite que los ejemplos sean numerados automáticamente a lo largo del documento.
- `\textit`: representa la traducción libre del ejemplo en castellano mediante tipografía cursiva. Esta convención es habitual en publicaciones lingüísticas para diferenciar claramente la traducción del texto original.
- `\begingl`: marca el inicio de una glosa interlineal. A partir de este punto se definen los diferentes niveles de análisis lingüístico asociados al ejemplo.
- `\gl a`: contiene la oración original tal y como aparece en la lengua maya analizada. Constituye el nivel superficial de representación y muestra la frase completa sin segmentación morfológica explícita.
- `\gl b`: almacena la segmentación morfológica de la oración. En esta línea se separan explícitamente los prefijos, raíces y sufijos que componen cada palabra, permitiendo visualizar la estructura interna de las formas lingüísticas.
- `\gl c`: contiene la glosa morfológica asociada a cada uno de los segmentos identificados en la línea anterior. Esta capa proporciona la interpretación gramatical de cada morfema mediante abreviaturas estandarizadas como COM (completivo), B3s (tercera persona singular de la serie B), A3p (tercera persona plural de la serie A), DET (determinante) o PL (plural).
- `\gl ft`: incorpora información adicional asociada al ejemplo. En este trabajo se utiliza para representar el análisis sintáctico generado por el sistema, indicando la función gramatical de determinados constituyentes de la oración.
- `\textsubscript`: permite representar etiquetas gramaticales en formato subíndice. En el ejemplo mostrado se emplea para indicar la categoría sintáctica asociada al constituyente analizado.

- `//`: actúa como delimitador de línea dentro del entorno de glosas. Este símbolo indica al paquete `expex` dónde finaliza cada nivel de análisis lingüístico.
- `\endgl`: señala el final de la glosa interlineal y cierra el bloque iniciado mediante `\begingl`.
- `\xe`: finaliza el ejemplo lingüístico y cierra la estructura abierta mediante `\ex`.

La comparación entre las distintas representaciones permite observar cómo una misma información lingüística puede expresarse simultáneamente en varios niveles de abstracción. El XML proporciona una estructura formal orientada al procesamiento automático, LaTeX ofrece una representación tipográfica especializada para documentación científica y el PDF compilado constituye el producto final destinado a la consulta humana. Esta separación entre contenido, estructura y presentación constituye uno de los principios fundamentales sobre los que se sustenta el sistema desarrollado en este trabajo.

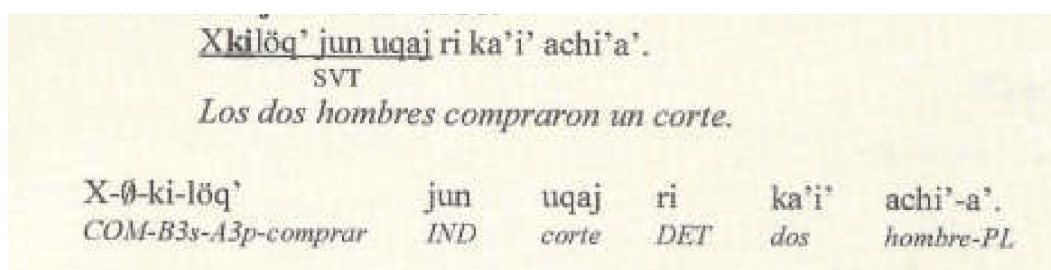


Figura 6.1.: Glosa interlineal en el documento de entrada (pdf)

Listado 6.1: Esquema XML refinado para la representación de glosas interlineales

```

1 <interlinear_gloss>
2   <parallel_phrase>
3     <original>Xkilöq' jun uqaj ri ka'i' achi'a'.</original>
4     <translation>Los dos hombres compraron un corte.</
      translation>
5   </parallel_phrase>
6   <morpheme_analysis>
7     <unit>
8       <form>X-</form>
9       <gloss>COM-</gloss>
10    </unit>
11    <unit>
12      <form>Ø-</form>
13      <gloss>B3s-</gloss>
14    </unit>
15    <unit>
16      <form>ki-</form>
17      <gloss>A3p-</gloss>
18    </unit>

```

```

19      <unit>
20          <form>löq'</form>
21          <gloss>comprar</gloss>
22      </unit>
23      <unit>
24          <form>jun</form>
25          <gloss>IND</gloss>
26      </unit>
27      <unit>
28          <form>uqaj</form>
29          <gloss>corte</gloss>
30      </unit>
31      <unit>
32          <form>ri</form>
33          <gloss>DET</gloss>
34      </unit>
35      <unit>
36          <form>ka'i'</form>
37          <gloss>dos</gloss>
38      </unit>
39      <unit>
40          <form>achi'-</form>
41          <gloss>hombre-</gloss>
42      </unit>
43      <unit>
44          <form>a'.</form>
45          <gloss>PL</gloss>
46      </unit>
47  </morpheme_analysis>
48  <syntactic_analysis>
49      <unit>
50          <form>Xkilöq'</form>
51          <gloss>SVT</gloss>
52      </unit>
53  </syntactic_analysis>
54 </interlinear_gloss>

```

Listado 6.2: Esquema Latex para la representación de glosas interlineales

```

1  \par\medskip%
2  \ex
3  \textit{Los dos hombres compraron un corte.} \\
4  \begin{gl}
5  \gla Xkilöq' jun uqaj ri ka'i' achi'a'. //
6  \glb X-Ø-ki-löq' jun uqaj ri ka'i' achi'-a'. //
7  \glc \textbf{COM-B3s-A3p-comprar} \textbf{IND} \textbf{corte} \
      \textbf{DET} \textbf{dos} \textbf{hombre-PL} //
8  \glft \textbf{Sintaxis:} [Xkilöq' jun uqaj]\textsubscript{SVT} //
9  \end{gl}

```

10 | \xe

Figura 6.2.: Glosa interlineal en el documento de salida a partir del Latex compilado (pdf)

(1) *Los dos hombres compraron un corte.*

Xkilöq' jun uqaj ri ka'i' achi'a'.

X-ø-ki-löq' jun uqaj ri ka'i' achi'-a'.

COM-B3s-A3p-comprar IND corte DET dos hombre-PL

Sintaxis: [Xkilöq' jun uqaj] svT

6.2. Diseño Experimental y Metodología de Evaluación

Para verificar el comportamiento estocástico del modelo multimodal y evaluar la resiliencia del sistema de validación sintáctica implementado en `xml_corrector.py`, se ha estructurado un protocolo experimental dividido en dos dimensiones complementarias: el barrido hiperparamétrico de la temperatura de generación y la comparativa entre diferentes arquitecturas de modelos desplegadas en la nube.

El corpus de validación comprende variantes dialectales e idiomáticas con distintos niveles de complejidad tipográfica, maquetación y ruido visual. Concretamente, se han seleccionado páginas pertenecientes a las gramáticas de Kaqchikel (páginas 172 y 205), Mam (páginas 90 y 231), K'iche' (páginas 52 y 56) y Q'anjob'al (páginas 44, 94 y 201).

El análisis de los datos se orienta a determinar cómo las características estructurales del documento fuente interactúan con las capacidades probabilísticas del modelo, evaluando la capacidad del sistema para generar documentos XML jerárquicamente válidos y compatibles con la especificación formal definida en el archivo DTD.

6.3. Análisis Empírico del Parámetro de Temperatura

El parámetro de temperatura (T) constituye el hiperparámetro crítico que gobierna la entropía y la distribución de probabilidad aplicada durante la generación de texto por parte del modelo. Su influencia resulta especialmente relevante en tareas de digitalización estructurada, donde existe un equilibrio delicado entre la reproducción fiel del contenido original y la capacidad de inferencia necesaria para reconstruir estructuras complejas o parcialmente degradadas.

Con el propósito de identificar el punto de equilibrio óptimo entre determinismo y flexibilidad generativa, se ha realizado un barrido sistemático de temperatura cubriendo ocho configuraciones diferenciadas:

$$T \in \{0,0, 0,1, 0,25, 0,5, 0,75, 1,0, 1,5, 2,0\}$$

Cada configuración ha sido evaluada mediante múltiples ejecuciones independientes sobre el conjunto de pruebas, permitiendo analizar tanto la calidad media de los resultados como la estabilidad estadística del proceso de generación. Esta metodología permite cuantificar la relación existente entre la variabilidad introducida por el modelo y la robustez del sistema de validación estructural desarrollado en los capítulos anteriores.

6.3.1. Comportamiento Global y No Monotonicidad del Sistema

El análisis agregado de las muestras revela una propiedad matemática fundamental: la relación entre la temperatura de generación y la fidelidad textual, medida mediante el porcentaje de similitud, no presenta un comportamiento lineal ni monotónico. La ordenación de los niveles térmicos en función de la media global obtenida es la siguiente:

$$T = 0,5 > T = 0,25 > T = 1,5 > T = 0,75 > T = 1,0 > T = 0,1 > T = 0,0 > T = 2,0$$

Este comportamiento pone de manifiesto que la precisión del sistema disminuye tanto en el extremo de máxima aleatoriedad ($T = 2,0$), con una similitud media del 92.14 % y una elevada dispersión reflejada en una desviación estándar de $\sigma = 8,51$, como en el extremo opuesto de determinismo absoluto ($T = 0,0$), cuya media global alcanza únicamente el 92.44 %.

Aunque la configuración correspondiente a $T = 0,5$ registra la mayor similitud media global (94.02 %) y la menor desviación estándar observada durante la experimentación ($\sigma = 5,06$), los resultados no permiten reducir la recomendación a un único valor aislado. En conjunto, la ventana comprendida entre $T = 0,1$ y $T = 0,75$ puede considerarse el rango más recomendable para el sistema, ya que agrupa configuraciones con un rendimiento elevado y diferencias relativamente reducidas entre sí. Esta observación sugiere que el sistema no depende de una temperatura exacta, sino de mantener un nivel de aleatoriedad bajo o moderado que proporcione cierta flexibilidad sin comprometer la estabilidad estructural.

6.3.2. El “Punto Dulce” (Sweet Spot) frente al Determinismo Absoluto

Contrariamente a la hipótesis habitual en tareas tradicionales de OCR y extracción automática de información, donde suele asumirse que una configuración completamente determinista ($T = 0,0$) minimiza los errores al seleccionar siempre el token más probable, los resultados experimentales obtenidos muestran que esta estrategia resulta subóptima cuando se trabaja con estructuras lingüísticas complejas.

La diferencia se aprecia claramente al comparar los resultados obtenidos bajo determinismo estricto con aquellos alcanzados dentro de la ventana térmica comprendida entre $T = 0,1$ y $T = 0,75$, identificada durante este trabajo como el rango de funcionamiento más recomendable del sistema.

- Caso Kaqchikel_205: bajo una configuración completamente determinista ($T = 0,0$), la similitud obtenida se reduce hasta el 92.66 %. Sin embargo, al incrementar ligeramente la temperatura hasta $T = 0,25$, el sistema alcanza un máximo de precisión del 97.76 %, lo que supone una mejora neta de 5.10 puntos porcentuales.
- Caso Q'anjob'al_201: este ejemplo constituye la evidencia más significativa de las limitaciones del determinismo absoluto. Con $T = 0,0$, la similitud registrada es del 70.63 %. Cuando la temperatura se ajusta a $T = 0,5$, el rendimiento aumenta hasta el 82.57 %, representando una mejora cercana a los 12 puntos porcentuales.

La explicación teórica de este fenómeno puede interpretarse en términos de lo que, dentro de este trabajo, se denomina *micro-flexibilidad cognitiva*. Durante el procesamiento de documentos analógicos heterogéneos, caracterizados por ruido visual, ausencia de metadatos textuales y estructuras complejas de glosas interlineales, una política completamente determinista obliga al modelo a permanecer dentro de trayectorias probabilísticas extremadamente rígidas.

Por el contrario, una temperatura baja o moderada, comprendida aproximadamente entre $T = 0,1$ y $T = 0,75$, introduce la flexibilidad necesaria para explorar interpretaciones alternativas de la estructura documental sin comprometer la coherencia global ni vulnerar las restricciones sintácticas impuestas por la gramática XML.

Los resultados obtenidos sugieren, por tanto, que la digitalización lingüística mediante modelos multimodales no debe abordarse como un problema puramente determinista. La incorporación controlada de aleatoriedad constituye un mecanismo eficaz para mejorar simultáneamente la fidelidad textual y la estabilidad estructural de la información generada.

6.3.3. Resiliencia Estructural e Insensibilidad Térmica en Páginas Estables

A pesar de las fluctuaciones observadas en las medias globales, el análisis desagregado de los resultados pone de manifiesto que la temperatura constituye un factor secundario para la mayoría de las muestras evaluadas. En concreto, siete de las nueve páginas analizadas (Kaqchikel_172, Mam_90, Mam_231, K'iche'_52, K'iche'_56, Q'anjob'al_44 y Q'anjob'al_94) presentan una sensibilidad extremadamente reducida frente a las variaciones del parámetro térmico, manteniendo rangos de oscilación inferiores o iguales a 1.61 puntos porcentuales.

El cálculo individual del rango de dispersión, definido como la diferencia entre el valor máximo y mínimo observado para cada página,

$$R = \text{máx} - \text{mín},$$

permite cuantificar con precisión esta estabilidad estructural.

Entre los casos más representativos destacan los siguientes:

- Q'anjob'al_44 ($R = 0,19$): constituye el ejemplo más extremo de estabilidad. La página mantiene una similitud constante del 96.10 % para todas las configuraciones

comprendidas entre $T = 0,0$ y $T = 0,75$. Únicamente a partir de $T \geq 1,0$ se observa una ligera transición hacia un valor estable del 95.91 %. Esta evolución presenta un comportamiento similar a una función escalón, lo que sugiere una elevada inmunidad frente al ruido estocástico introducido por el modelo.

- Q'anjob'al_94 ($R = 0,35$): muestra un patrón análogo. La similitud permanece prácticamente invariable en torno al 99.92 % para casi todas las temperaturas analizadas, registrando pequeñas desviaciones únicamente en $T = 0,5$ y $T = 2,0$, donde alcanza el 99.57 %.
- K'iche' _52 ($R = 0,56$), Mam_231 ($R = 0,61$) y K'iche' _56 ($R = 0,68$): presentan igualmente líneas de tendencia prácticamente horizontales. Resulta especialmente interesante el caso de K'iche' _56, cuya similitud permanece estabilizada alrededor del 88.90 % independientemente de la temperatura utilizada. Este comportamiento sugiere la existencia de una limitación estructural específica de la página, posiblemente asociada a elementos complejos de maquetación, glifos ambiguos o construcciones lingüísticas que exceden las capacidades de interpretación del sistema. En consecuencia, incrementar o reducir la aleatoriedad del modelo no produce mejoras apreciables.

La evidencia experimental obtenida permite concluir que la tipología documental, la calidad de digitalización y la complejidad estructural de la página original constituyen el principal factor limitante del proceso de extracción. Su influencia resulta significativamente superior a la derivada de la configuración térmica del modelo.

Desde esta perspectiva, la temperatura actúa como un factor de ajuste fino cuya capacidad de mejora depende de la existencia previa de ambigüedades interpretativas. Cuando el documento presenta una estructura limpia, una disposición tipográfica estable y texto claramente identificable, el sistema converge de forma consistente hacia niveles de similitud próximos al 99 %. Por el contrario, las páginas con ruido visual, estructuras complejas o dificultades inherentes de maquetación permanecen confinadas a techos de rendimiento relativamente estables, independientemente del nivel de creatividad asignado al modelo generativo.

6.3.4. Correlación entre la Complejidad del Contenido y la Sensibilidad Térmica

Con el objetivo de profundizar en la relación entre la dificultad intrínseca del documento y el impacto de la creatividad introducida por el modelo, se ha definido una métrica específica denominada factor diferencial de creatividad (Δ). Esta magnitud cuantifica la diferencia de rendimiento entre entornos de alta creatividad y configuraciones cercanas al determinismo.

Formalmente, el factor se define como:

$$\Delta_{\text{creatividad}} = \text{Media}(T = 1,5, T = 2,0) - \text{Media}(T = 0,0, T = 0,1)$$

Un valor positivo de Δ indica que el aumento de la creatividad beneficia el proceso de extracción, mientras que un valor negativo implica que la mayor aleatoriedad degrada el rendimiento y favorece la aparición de errores estructurales o textuales.

Al relacionar esta métrica con la dificultad media de cada página, medida mediante su porcentaje de similitud global, se obtiene la distribución mostrada en la Tabla 6.1.

Tabla 6.1.: Relación entre dificultad documental e impacto de la creatividad

Página	Similitud media	Δ	Impacto
Q'anjob'al_201	75.32 %	+1.69	Beneficioso
K'iche'_56	88.90 %	+0.63	Beneficioso
Kaqchikel_205	93.43 %	-1.42	Perjudicial
Kaqchikel_172	93.47 %	-0.05	Neutro
Mam_90	94.02 %	-0.30	Perjudicial
Q'anjob'al_44	96.03 %	-0.19	Perjudicial
K'iche'_52	98.32 %	-0.14	Perjudicial
Mam_231	98.64 %	-0.30	Perjudicial
Q'anjob'al_94	99.83 %	-0.17	Perjudicial

Los resultados obtenidos permiten extraer varias observaciones relevantes acerca del comportamiento del sistema.

En primer lugar, se aprecia una tendencia de compensación mediante creatividad. Las dos páginas con menor rendimiento global, Q'anjob'al_201 y K'iche'_56, son precisamente las únicas que presentan valores positivos de Δ . Este comportamiento sugiere que, cuando el modelo se enfrenta a documentos con elevados niveles de ambigüedad visual o dificultades de interpretación estructural, una mayor libertad probabilística favorece la reconstrucción contextual de información incompleta o deteriorada.

Por el contrario, en aquellas páginas cuya similitud media supera aproximadamente el 94 %, la creatividad elevada no aporta beneficios observables. Los valores de Δ se mantienen sistemáticamente en una banda reducida comprendida entre $-0,05$ y $-0,30$, indicando que el incremento de temperatura introduce pequeñas alteraciones que terminan perjudicando la calidad de la extracción. En documentos con buena calidad visual y estructuras claramente identificables, la tarea se aproxima a un problema de transcripción directa donde la exploración probabilística resulta innecesaria.

Un aspecto especialmente interesante lo constituye el caso de Kaqchikel_205. A pesar de presentar una dificultad intermedia-alta, con una similitud media del 93.43 %, el aumento de creatividad provoca una degradación significativa de 1.42 puntos porcentuales. Este comportamiento rompe la tendencia general observada y sugiere que no todas las formas de complejidad documental se benefician por igual de la aleatoriedad. Es posible que la dificultad de esta página esté asociada a estructuras altamente jerarquizadas, como tablas de conjugación verbal o disposiciones sintácticas rígidas, cuya correcta interpretación requiere estabilidad estructural más que capacidad inferencial.

En conjunto, los resultados indican que el efecto de la temperatura depende no solo del grado de dificultad de la página, sino también de la naturaleza específica de dicha

dificultad. Mientras que los problemas asociados a ruido visual o información incompleta pueden beneficiarse de una mayor flexibilidad generativa, las estructuras lingüísticas altamente organizadas tienden a degradarse cuando aumenta la variabilidad probabilística.

A la luz de esta evidencia, la ventana térmica comprendida entre $T = 0,1$ y $T = 0,75$ se consolida como la configuración más equilibrada para un sistema generalista de digitalización de gramáticas de la ALMG. Este intervalo proporciona suficiente flexibilidad para mejorar la extracción en los documentos más complejos, manteniendo simultáneamente la estabilidad necesaria para evitar degradaciones en aquellas páginas que presentan una estructura clara y fácilmente interpretable.

6.4. Evaluación Comparativa de Arquitecturas Multimodales (Modelos)

Una vez determinado el impacto del entorno térmico sobre el proceso de inferencia, la segunda dimensión experimental de este estudio se centra en la comparación directa de diferentes arquitecturas multimodales disponibles a través de la API de Google AI.

Con el fin de aislar las capacidades intrínsecas de reconocimiento visual, razonamiento documental y estructuración sintáctica de cada modelo, todas las pruebas se ejecutaron utilizando una temperatura constante de $T = 1,0$. Esta parametrización permite minimizar el efecto de la configuración térmica y atribuir las diferencias observadas exclusivamente a las características propias de cada arquitectura.

El análisis incluye cinco modelos pertenecientes a distintas generaciones de la familia Gemini:

- `gemini-2.5-flash`
- `gemini-2.5-flash-lite`
- `gemini-3-flash-preview`
- `gemini-3.1-flash-lite`
- `gemini-3.5-flash`

Los resultados obtenidos muestran diferencias significativas tanto en precisión como en estabilidad, especialmente cuando los modelos se enfrentan a documentos con una elevada complejidad estructural.

6.4.1. Dominancia Sistémica de la Arquitectura Avanzada (`gemini-3.5-flash`)

Los resultados experimentales identifican de forma clara a `gemini-3.5-flash` como la arquitectura con mejor rendimiento global dentro del conjunto analizado.

Este modelo alcanza una similitud media del 95.69 %, la más elevada de todo el estudio, y presenta simultáneamente la menor dispersión estadística, con una desviación estándar de apenas

$$\sigma = 3,59.$$

La combinación de una elevada precisión media y una baja variabilidad constituye una evidencia sólida de la robustez operativa del modelo. No solo obtiene mejores resultados absolutos, sino que además mantiene un comportamiento consistente entre documentos de naturaleza muy diversa.

La superioridad de la arquitectura queda reforzada al comprobar que obtiene la mejor puntuación en seis de las nueve páginas incluidas en el corpus experimental.

Las diferencias se vuelven especialmente relevantes en aquellos documentos catalogados como escenarios de alta complejidad estructural.

- Caso K'iche'_56: **gemini-3.5-flash** alcanza una similitud del 96.80 %, superando en 5.13 puntos porcentuales al segundo modelo mejor clasificado, **gemini-3-flash-preview**, que obtiene un 91.67 %. Esta diferencia constituye la mayor separación observada entre dos arquitecturas durante todo el estudio y pone de manifiesto la importancia de la capacidad de razonamiento documental cuando el sistema debe interpretar estructuras complejas.
- Caso Q'anjob'al_201: considerada la página más difícil del conjunto de pruebas, **gemini-3.5-flash** lidera nuevamente la clasificación con una similitud del 87.96 %. Se trata de la única arquitectura capaz de aproximarse de forma consistente a niveles de precisión compatibles con una digitalización prácticamente autónoma en un entorno de elevada dificultad.

Estos resultados sugieren que el incremento de capacidad computacional y profundidad de razonamiento incorporado en las arquitecturas más avanzadas no solo mejora el reconocimiento textual, sino que también incrementa significativamente la capacidad del modelo para interpretar jerarquías documentales complejas, glosas interlineales y estructuras lingüísticas altamente organizadas.

La evidencia empírica obtenida permite concluir que **gemini-3.5-flash** representa la opción más adecuada para tareas de digitalización lingüística avanzada, especialmente cuando el objetivo es preservar simultáneamente la fidelidad textual y la estructura semántica de documentos especializados.

6.4.2. El Comportamiento Paradójico de los Modelos de Escala Reducida (Lite)

La evaluación de **gemini-2.5-flash-lite** revela un comportamiento especialmente interesante desde el punto de vista de la ingeniería de *prompts* y de la selección de arquitecturas para tareas de digitalización documental.

A nivel agregado, este modelo obtiene los resultados más modestos de todo el estudio, registrando una similitud media global del 84.94 % y la mayor dispersión estadística observada entre todas las arquitecturas evaluadas, con una desviación estándar de

$$\sigma = 10,71.$$

Estos valores reflejan una elevada sensibilidad a las características específicas del documento procesado. Sin embargo, un análisis desagregado muestra que esta arquitectura no puede considerarse simplemente como un modelo de bajo rendimiento.

Los resultados evidencian una fuerte dependencia respecto a la complejidad intrínseca de la página analizada. En documentos con una estructura sencilla, escasa densidad de glosas y una maquetación clara, el modelo ofrece resultados altamente competitivos. Un ejemplo representativo es la página Q'anjob'al_94, donde alcanza una similitud del 100.00 %, igualando el mejor resultado obtenido durante toda la experimentación y compartiendo dicha posición con `gemini-2.5-flash`.

No obstante, esta capacidad se degrada rápidamente cuando aumenta la complejidad documental. En páginas caracterizadas por estructuras densas, presencia de ruido visual, escaneados de baja calidad o distribuciones espaciales complejas, el modelo experimenta una caída significativa de rendimiento. El caso más representativo corresponde a Kaqchikel_172, donde la similitud desciende hasta el 70.14 %.

Esta marcada asimetría sugiere que las versiones reducidas presentan limitaciones para mantener representaciones internas suficientemente robustas cuando la tarea requiere combinar simultáneamente reconocimiento visual, razonamiento estructural y generación jerárquica de XML. Como consecuencia, su utilización resulta especialmente adecuada en escenarios donde los documentos han sido previamente limpiados, normalizados o presentan una estructura relativamente lineal.

6.4.3. Elasticidad de la Brecha Intermodelo en Función de la Complejidad del Layout

Uno de los hallazgos más relevantes de esta experimentación es que la importancia de seleccionar una arquitectura específica depende directamente de la complejidad del documento procesado.

Los resultados muestran que la diferencia entre el mejor y el peor modelo no permanece constante, sino que aumenta progresivamente a medida que disminuye la legibilidad o aumenta la complejidad estructural de la página.

En los documentos más sencillos, las diferencias entre arquitecturas son prácticamente insignificantes. El ejemplo más representativo es la página Q'anjob'al_94, donde la distancia entre el peor y el mejor resultado es de únicamente 1.44 puntos porcentuales, oscilando entre el 98.56 % y el 100.00 %.

Desde una perspectiva práctica, este comportamiento indica que en tareas de baja complejidad la elección del modelo tiene un impacto limitado sobre la calidad final de la digitalización. En estos escenarios resulta razonable priorizar factores operativos como el coste por token, el tiempo de respuesta o el consumo de recursos computacionales.

La situación cambia radicalmente en los documentos más complejos. En la página Q'anjob'al_201, identificada como la muestra de mayor dificultad del corpus, la diferencia entre la mejor y la peor arquitectura se amplía hasta 17.65 puntos porcentuales, abarcando desde un mínimo del 70.31 % hasta un máximo del 87.96 %.

Esta expansión de la brecha de rendimiento evidencia que la complejidad documental actúa como un factor amplificador de las diferencias arquitectónicas. Cuando el modelo debe interpretar estructuras lingüísticas densas, glosas interlineales complejas o disposiciones tipográficas ambiguas, las capacidades de razonamiento multimodal y representación jerárquica adquieren una importancia decisiva.

Los resultados permiten concluir que la selección de la arquitectura es prácticamente irrelevante para documentos sencillos, pero se convierte en un factor crítico conforme aumenta la complejidad del material analizado. En estos contextos, la diferencia entre una arquitectura avanzada y una versión reducida puede determinar si el sistema genera una representación XML válida o si, por el contrario, se producen errores estructurales que comprometen la utilidad del documento digitalizado.

6.4.4. No Linealidad Genérica del Rendimiento frente al Nomenclátor de Versión

Los resultados experimentales obtenidos permiten cuestionar una hipótesis frecuentemente asumida en el ámbito de los Modelos de Lenguaje de Gran Tamaño: que una versión más reciente de una arquitectura implica necesariamente una mejora proporcional en el rendimiento de tareas especializadas.

La evidencia recopilada durante este estudio demuestra que la numeración generacional de un modelo no constituye un predictor fiable de su desempeño en procesos de digitalización lingüística estructurada. Por el contrario, el rendimiento observado depende de factores más complejos relacionados con la arquitectura interna, los procedimientos de entrenamiento y los mecanismos específicos de razonamiento multimodal incorporados en cada versión.

Un primer ejemplo de esta no linealidad se observa al comparar **gemini-3-flash-preview** con **gemini-2.5-flash**. Aunque el primero pertenece nominalmente a una generación posterior, registra una similitud media global inferior. Mientras **gemini-2.5-flash** alcanza una media del 93.37 %, **gemini-3-flash-preview** obtiene un 92.44 %.

Un comportamiento similar se aprecia en **gemini-3.1-flash-lite**. A pesar de tratarse de una variante ligera, sus resultados se mantienen próximos a los obtenidos por **gemini-2.5-flash**, alcanzando niveles de precisión que superan las expectativas habitualmente asociadas a modelos de menor tamaño.

La naturaleza no lineal de esta relación se manifiesta con especial claridad al analizar casos individuales. El ejemplo más representativo corresponde a la página Kaqchikel_205. En este documento, **gemini-3-flash-preview** alcanza una similitud del 94.70 %, superando a **gemini-3.5-flash**, que registra un 92.32 %.

Este resultado constituye una inversión local de la jerarquía global observada anteriormente y demuestra que la superioridad estadística de un modelo no garantiza necesariamente un mejor comportamiento en todos los escenarios posibles. Determinadas

configuraciones documentales pueden alinearse mejor con los sesgos internos o las capacidades específicas de arquitecturas concretas, produciendo resultados superiores a los obtenidos por modelos de mayor capacidad general.

Desde una perspectiva metodológica, esta observación posee implicaciones relevantes para el diseño de sistemas de digitalización lingüística. Los resultados sugieren que la selección del modelo no debería realizarse exclusivamente atendiendo a su posición dentro de una determinada familia de versiones, sino considerando también la naturaleza estructural del corpus que se desea procesar.

En consecuencia, la evidencia empírica obtenida respalda la adopción de arquitecturas flexibles capaces de conmutar dinámicamente entre diferentes modelos según las características del documento de entrada. Este enfoque permite aprovechar las fortalezas específicas de cada arquitectura y evita depender de la hipótesis simplificadora de que una versión más reciente constituye siempre la mejor alternativa para cualquier tarea de digitalización estructurada.

En definitiva, la evolución cronológica de los modelos no sigue una relación lineal con el rendimiento observado. La interacción entre complejidad documental, capacidades multimodales y sesgos de entrenamiento genera un escenario donde la elección óptima depende del contexto particular de cada documento, reforzando la necesidad de evaluar empíricamente cada arquitectura antes de integrarla en un entorno de producción.

7. Conclusiones y trabajo futuro

Este capítulo final ofrece una visión retrospectiva sobre el desarrollo del proyecto, evalúa el cumplimiento de las metas propuestas y proyecta posibles líneas de investigación que permitan escalar el sistema a nuevos horizontes tecnológicos.

7.1. Síntesis del trabajo desarrollado

En el marco de este Trabajo de Fin de Máster, hemos diseñado e implementado un ecosistema tecnológico integral orientado a la preservación digital de lenguas mayas minorizadas. La investigación ha abordado con éxito el desafío de transformar documentos estáticos y fotografías de libros de la Academia de Lenguas Mayas de Guatemala (ALMG) en datos digitales inteligentes y estructurados.

El trabajo desarrollado se ha materializado en un pipeline modular en Python que orquesta cuatro procesos críticos:

- Procesamiento multimodal: La conversión de documentos PDF heterogéneos en entradas visuales de alta calidad.
- Inferencia Inteligente: El uso de modelos de lenguaje de gran tamaño (Gemini 1.5, 2.5...) mediante técnicas de Few-Shot Learning para la extracción semántica de gramáticas complejas.
- Garantía de calidad estructural: La implementación de una capa de validación basada en un Documento de Definición de Tipo (DTD) y un corrector heurístico que subsana las limitaciones sintácticas de la IA.
- Exportación académica: La traslación automática de los datos a código LaTeX, permitiendo una reconstrucción tipográfica profesional de las obras originales.

7.1.1. Conclusiones del experimento con diferentes temperaturas

Más allá del desarrollo de software, este trabajo ha aportado un valor investigador significativo mediante un estudio empírico de la temperatura del modelo. Los experimentos han demostrado una serie de conclusiones que mostramos a continuación:

- El rango comprendido entre $T = 0,1$ y $T = 0,75$ es la configuración globalmente recomendable para el sistema evaluado sobre este corpus. Aunque $T = 0,5$ obtiene la mayor media de similitud (94,02) y la menor dispersión entre páginas ($\sigma = 5,06$), las diferencias entre las temperaturas bajas y moderadas no son lo suficientemente

amplias como para justificar una recomendación limitada a un único valor. Por ello, resulta más prudente considerar una ventana de trabajo estable, en la que el sistema mantiene un rendimiento alto sin depender de una temperatura exacta.

- La temperatura no es el factor determinante del rendimiento global del sistema. La variabilidad entre páginas —de hasta 30 puntos— supera con creces el efecto de la temperatura —cuya influencia máxima sobre la media global es de 1,88 puntos—. El principal determinante del CER es la complejidad intrínseca de cada página.
- La temperatura extrema alta ($T = 2,0$) es la peor configuración del rango evaluado, con la menor media global (92,14) y la mayor dispersión ($\sigma = 8,51$). Su uso no ofrece ventajas sostenidas en ninguna página del corpus.
- El determinismo absoluto ($T = 0$) tampoco es óptimo. La intuición de que la mínima temperatura maximiza la precisión no se confirma en los datos: $T = 0$ produce la segunda menor media global (92,44) y una dispersión elevada ($\sigma = 8,45$). Para esta tarea, el modo puramente determinista es subóptimo.
- La sensibilidad a la temperatura es función de la dificultad de la página. Siete de las nueve páginas son prácticamente insensibles al parámetro (rango ¡2 puntos). La temperatura solo produce variaciones de magnitud operativa en páginas que ya presentan dificultades para el sistema.
- La relación entre temperatura y CER no es monotónica. El ordenamiento de temperaturas por media global no sigue ningún patrón predecible de creatividad creciente. En lugar de existir un valor óptimo único, los resultados apuntan a una ventana recomendable entre $T = 0,1$ y $T = 0,75$, mientras que los extremos del rango, especialmente $T = 0$ y $T = 2,0$, son menos adecuados en términos de media y estabilidad.

Conclusiones con reservas:

- En páginas de alta dificultad, la temperatura alta tiende a producir mejores resultados que la temperatura baja. Las dos páginas más difíciles (Qanjobal_201 y Kiche_56) muestran un efecto neto positivo de la temperatura alta ($\Delta = +1,69$ y $+0,63$). Sin embargo, la existencia de Kaqchikel_205 como excepción significativa ($\Delta = -1,42$ con dificultad intermedia) impide elevar esta tendencia a conclusión general. Se trata de una señal observable, no de una regla.

Limitaciones del estudio:

- La ausencia de réplicas por condición impide realizar pruebas de significancia estadística. Las diferencias inferiores a 2 puntos porcentuales deben considerarse indicativas.

- El corpus de nueve páginas, aunque diverso, es reducido. Las conclusiones tienen validez interna para el conjunto evaluado y requieren verificación en corpus ampliados antes de generalizarse.
- No es posible determinar a partir de los datos disponibles la causa concreta del bajo rendimiento sistemático de Qanjobal_201. Un análisis cualitativo de esta página es recomendable para identificar sus factores limitantes y orientar futuras mejoras del sistema.

7.1.2. Conclusiones del experimento con diferentes modelos

Además del estudio sobre temperaturas también contamos con el experimento de los diferentes modelos de gemini. Las conclusiones obtenidas han sido las siguientes:

1. gemini-3.5-flash es el modelo dominante en todos los sentidos.

Es simultáneamente el de mayor media (95.69) y el de menor desviación estándar (3.59). Que ambas métricas apunten al mismo modelo es un resultado robusto: no solo rinde más, sino que lo hace de forma consistente. Además gana en 6 de las 9 páginas.

2. gemini-2.5-flash-lite tiene un comportamiento paradójico.

Es el peor modelo en media global (84.94) y el más variable ($\sigma = 10,71$), pero alcanza un CER perfecto de 100.00 en Qanjobal_94 junto con gemini-2.5-flash. Esto descarta que sea un modelo uniformemente deficiente: es extremadamente sensible a la dificultad del contenido. Cuando la página es fácil, compite con el mejor; cuando es difícil, se desploma hasta 70.14.

3. La elección del modelo importa mucho más en páginas difíciles.

En Qanjobal_94 (la página más fácil), el rango entre el mejor y el peor modelo es de 1.44 puntos (98.56–100.00). En Qanjobal_201 (la más difícil), ese rango es de 17.65 puntos (70.31–87.96). En términos prácticos: para contenido sencillo da igual qué modelo usar; para contenido complejo, la elección es determinante.

4. gemini-3.5-flash domina especialmente en las páginas más problemáticas.

En Kiche_56, obtiene 96.80 frente a un máximo de 91.67 del segundo mejor modelo, una brecha de 5.13 puntos, la mayor observada en cualquier página. En Qanjobal_201 también lidera con 87.96. Este modelo no solo gana en la media, sino que es donde más se necesita.

5. La generación del modelo no predice linealmente el rendimiento.

gemini-3-flash-preview (generación “3”) queda por debajo de gemini-2.5-flash (generación “2.5”) en media global (92.44 vs. 93.37). Y gemini-3.1-flash-lite obtiene resultados comparables a gemini-2.5-flash a pesar de ser una versión “lite”. El número de versión no es un predictor fiable del CER en este estudio.

6. gemini-3.5-flash no gana en todas las páginas sin excepción.

En Kaqchikel_205, gemini-3-flash-preview obtiene 94.70 frente a 92.32 de gemini-3.5-flash. Es el único caso en que otro modelo supera al dominante, lo que impide afirmar que gemini-3.5-flash sea universalmente óptimo.

Lo que no se puede afirmar:

- Con 9 páginas no hay suficiente muestra para establecer significancia estadística.
- Tampoco es posible saber si las diferencias entre modelos se deben a la arquitectura, al entrenamiento con estas lenguas concretas, o a otros factores.
- La comparación solo es válida a temperatura 1.0.

7.2. Grado de cumplimiento de los objetivos

Tras finalizar el desarrollo y la experimentación, se confirma que el proyecto ha cumplido satisfactoriamente con el objetivo general y los objetivos específicos definidos en la introducción de esta memoria. A continuación, se detalla el grado de consecución de cada uno de ellos:

- Objetivo 1: Implementación del pipeline de procesamiento.

Estado: Alcanzado. Se ha codificado una arquitectura modular en Python que integra con éxito la ingesta de PDFs, la interacción con la API de Gemini y la gestión de archivos de salida. El sistema es plenamente funcional y reproducible.

- Objetivo 2: Definición de la estructura de datos (DTD).

Estado: Alcanzado. Se ha diseñado una gramática formal (`gramatica.dtd`) que encapsula la complejidad de los textos lingüísticos mayas, asegurando que la información extraída mantenga su jerarquía lógica.

- Objetivo 3: Desarrollo de herramientas de validación y corrección.

Estado: Alcanzado. Los módulos `xml_corrector.py` y `xml_validator.py` han demostrado ser fundamentales para subsanar la inestabilidad de los LLMs, logrando que el 100 % de las salidas aprobadas por el sistema sean estructuralmente válidas.

- Objetivo 4: Estudio empírico de la temperatura.

Estado: Alcanzado. Se ha realizado una experimentación sistemática que ha permitido identificar el intervalo de $T = 0,1$ a $T = 0,75$ como el rango de funcionamiento más recomendable, aportando una conclusión técnica de alto valor para futuros trabajos en el área.

- **Objetivo 5: Generación de documentación en LaTeX.**

Estado: Alcanzado. El script `converter.py` automatiza la traslación de los datos estructurados a archivos `.tex` de alta calidad, permitiendo la reconstrucción visual de las gramáticas de la ALMG bajo estándares académicos.

- **Objetivo 6: Evaluación de la portabilidad a modelos locales.**

Estado: Alcanzado (Nivel Exploratorio). Se ha realizado un análisis del estado del arte (Capítulo 2.4) identificando a Mistral OCR y Granite-Docling como las alternativas locales más viables para mitigar la dependencia de APIs externas detectadas durante el desarrollo.

Conclusión del Objetivo General:

El sistema desarrollado constituye una solución robusta para la digitalización y estructuración de lenguas minorizadas. Se ha logrado transformar fuentes documentales analógicas en activos digitales de alta precisión (90-100 %), cumpliendo con los estándares de calidad tipográfica y técnica requeridos para un TFM de Máster en Inteligencia Artificial.

7.3. Lecciones aprendidas y desafíos técnicos

El desarrollo de este proyecto ha supuesto un proceso de aprendizaje continuo, enfrentando retos que van más allá del desarrollo de software y que tocan la realidad operativa de la Inteligencia Artificial en 2026.

7.3.1. La fragilidad de la dependencia de APIs externas

El desafío técnico más crítico fue la inestabilidad de las condiciones de servicio de Google AI Studio. Durante el proyecto, la cuota de uso sufrió una degradación drástica, pasando de 1.500 peticiones diarias a solo 20.

Lección: Esta limitación forzó una transición desde un enfoque de “procesamiento masivo” hacia una “economía de tokens” y de tiempo. Se aprendió que en investigación es vital contar con un Test Set optimizado para no agotar los recursos en pruebas fallidas. Asimismo, subraya la necesidad de avanzar hacia el despliegue de modelos locales (como se propone en el trabajo futuro) para garantizar la soberanía tecnológica del proyecto.

7.3.2. La IA como motor, la ingeniería como cinturón de seguridad

Una lección fundamental ha sido la confirmación de que, en tareas de digitalización académica, no se puede confiar ciegamente en la salida de un LLM, por avanzado que sea.

Lección: La verdadera robustez del sistema no reside en el modelo Gemini en sí, sino en la capa de validación externa (DTD) y en el post-procesamiento heurístico

(`xml.corrector.py`). Se ha aprendido que la IA es excelente para la comprensión contextual, pero la programación tradicional sigue siendo indispensable para garantizar la perfección sintáctica necesaria en formatos como XML o LaTeX.

7.3.3. Complejidad lingüística maya

El procesamiento de glosas interlineales y caracteres especiales puso a prueba los límites del modelo.

Lección: Se aprendió que el Few-Shot Learning es la herramienta más potente para “enseñar” a una IA reglas de una lengua minorizada en pocos segundos. Sin estos ejemplos previos, el modelo tendía a ignorar la estructura vertical de las glosas, lo que demuestra la importancia de la curación de datos por parte del investigador.

7.4. Líneas de investigación futura

El sistema desarrollado en este TFM sienta las bases para un ecosistema de digitalización lingüística mucho más amplio. Se proponen las siguientes líneas de trabajo para dar continuidad a la investigación:

7.4.1. Migración a modelos locales y soberanía tecnológica

La línea prioritaria de investigación futura es la sustitución o apoyo de la API de Gemini por modelos de pesos abiertos (*Open Weights*) que puedan ejecutarse en infraestructura local.

- Implementación de Mistral OCR y DeepSeek-OCR: Evaluar el rendimiento de estos modelos especializados en visión documental para eliminar la dependencia de cuotas de terceros y costes por token.
- Despliegue de modelos ligeros: Explorar el uso de IBM Granite-Docling (258M) en entornos de computación restringida (como Google Colab o servidores departamentales), lo que permitiría el procesamiento masivo de las gramáticas de forma ininterrumpida.

7.4.2. Expansión del corpus lingüístico

Una vez validado el pipeline en Kaqchikel, K'iche', Mam y Q'anjob'al, el siguiente paso natural es extender el proceso a las restantes lenguas de la Academia de Lenguas Mayas de Guatemala.

- Inclusión de lenguas con menos recursos: Aplicar la metodología al Q'eqchi' y Poqomchi', entre otros, para verificar si la capacidad de Transfer Learning del sistema se mantiene constante en toda la familia maya.

7.4.3. Desarrollo de una plataforma de consulta lingüística

El contenido estructurado en XML generado por este sistema permite el desarrollo de herramientas de nivel superior:

- Buscador semántico: Crear una base de datos que permita a lingüistas realizar búsquedas complejas (ej. “buscar todos los ejemplos de verbos transitivos en Kaq-chikel con su glosa correspondiente”).
- Generación automática de diccionarios: Utilizar las glosas extraídas para alimentar automáticamente bases de datos léxicas y motores de traducción automática.

7.4.4. Optimización del pipeline mediante RAG

Implementar una arquitectura de Generación Aumentada por Recuperación (RAG) para la selección dinámica de ejemplos few-shot. En lugar de usar ejemplos estáticos, el sistema podría buscar en una base de datos de páginas ya validadas aquellas que tengan un layout similar a la página actual, optimizando así la precisión de la IA en tiempo real.

7.4.5. Colaboración institucional y difusión de datos

Continuar con las gestiones iniciadas por la tutoría académica para establecer acuerdos con la ALMG que permitan, en un futuro, la liberación de los corpus estructurados bajo licencias abiertas, contribuyendo de forma definitiva a la digitalización de las lenguas mayas.

Personalmente, este proyecto ha sido una oportunidad para profundizar en tecnologías y conceptos que no formaban parte de mi conocimiento previo. Desde el aprendizaje de nuevas herramientas como la integración de APIs de LLMs hasta la adopción de buenas prácticas como el uso de estándares de Python o el registro de logs, he adquirido habilidades que serán valiosas en futuros proyectos. Además, la experiencia de trabajar en un proyecto de esta envergadura ha reforzado mi capacidad para planificar, organizar y resolver problemas de manera eficiente.

En conclusión, este proyecto no solo ha cumplido con sus objetivos iniciales, sino que también ha sentado las bases para futuras mejoras y desarrollos. Espero que el trabajo realizado contribuya al avance de aplicaciones similares y sirva como referencia para otros desarrolladores e investigadores.

Bibliografía

- A Bhat, A., Naganna, S., Haq, S., Khatri, P., Tamataam, K. C. R., Arun, N., Chhaya, N., & Bhattacharya, P. (2026). SAVIOR: Sample-efficient Adaptation of Vision-Language Models for OCR Representation. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops*, 719-728.
- Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised Cross-lingual Representation Learning at Scale. En D. Jurafsky, J. Chai, N. Schluter & J. Tetreault (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440-8451). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.747>
- Ding, Y., Luo, S., Dai, Y., Jiang, Y., Li, Z., Sun, Q., Martin, G., Liu, W., & Peng, Y. (2026). A Survey on MLLM-based Visually Rich Document Understanding: Methods, Challenges, and Emerging Trends. <https://arxiv.org/abs/2507.09861>
- Echavarria, A., & Le Meur, M. (2025, septiembre). *CER y WER: métricas de evaluación de modelos de reconocimiento automático de manuscritos e impresos antiguos* (inf. téc.). Consortium-HN ARIANE - Analyses, Recherches, Intelligence Artificielle et Nouvelles Editions numériques. <https://hal.science/hal-05267874>
- Fu, L., et al. (2025). OCRBench v2: An Improved Benchmark for Evaluating Large Multimodal Models on Visual Text Localization and Reasoning.
- Shen, A., Chen, T., Sun, Y., Jiang, F., Feng, Y., Hu, K., & Liu, M. (2025). Vision-driven Adaptive Decoding for Document Understanding: A Dual-Path Visual-Language and OCR Framework with Prompt-aware Task Routing. *2025 9th International Conference on Vision, Image and Signal Processing (ICVISIP)*, 1-5. <https://doi.org/10.1109/ICVISIP68610.2025.11451708>
- Singh, D., Bansal, A., & Bansal, N. (2015). A Review on Optical Character Recognition Using Various Techniques. <https://api.semanticscholar.org/CorpusID:201633108>
- Wei, H., Sun, Y., & Li, Y. (2026). DeepSeek-OCR 2: Visual Causal Flow. <https://arxiv.org/abs/2601.20552>
- Xu, J., Li, Z., Chen, W., Wang, Q., Gao, X., Cai, Q., & Ling, Z. (2024). On-Device Language Models: A Comprehensive Review. <https://arxiv.org/abs/2409.00088>

A. Anexo I. Esquemas en xml

Listado A.1: Kaqchikel XML generado por el LLM a partir de la imagen de la página original

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <document>
3   <page number="203">
4     <heading>... / .. / .... Oración compleja</heading>
5     <line>La oración compleja se forma con más de una cláusula
6       , una de ellas es la principal y</line>
7     <line>otra la subordinada. La idea que expresan las clá
8       usulas subordinadas siempre dependen</line>
9     <line>de la cláusula principal, su posición es siempre
10      después de ésta. Una cláusula subordinada</line>
11     <line>se introduce por medio de subordinadores, aunque hay
12      excepciones. Son oraciones</line>
13     <line>complejas si dentro de la cláusula principal se
14      encuentran las siguientes cláusulas</line>
15     <line>subordinadas:</line>
16
17     <line>Cláusula relativa</line>
18     <line>Cláusula adjunta o adverbial</line>
19     <line>Cláusula de complemento</line>
20
21     <heading>... / .. / .... / . Clausula relativa</heading>
22     <line>La clausula relativa modifica a sustantivos,
23       describe alguna característica, por eso</line>
24     <line>funciona como adjetivo. La clausula relativa se
25       introduce por los subordinadores: ri,</line>
26     <line>re, la, achoq rik'in y achoq chi re o simplemente
27       subordinador. Estas particulas se</line>
28     <line>conocen con los nombres de pronombre relativo o
29       relativizador. Si modifica al sujeto u</line>
30     <line>objeto, se introduce con las particulas: ri, re, la.
31       Si modifica al instrumento o compania,</line>
32     <line>la clausula relativa se introducen con la frase
33       achoq rik'in; y si la clausula no se</line>
34     <line>introduce por subordinador, la clausula relativa se
35       escribe entre comas. Ejemplos:</line>
```

```

27 <line>Modificando al sujeto:</line>
28 <interlinear_gloss>
29   <parallel_phrase>
30     <original>Ri ix\"{o}q, ri xkos, xb'eruloq'o' xkoya
31     ' .</original>
32     <translation>La mujer que se cansó fue a comprar
33     tomate</translation>
34   </parallel_phrase>
35   <morpheme_analysis>
36     <unit>
37       <form>Ri</form>
38       <gloss>DET</gloss>
39     </unit>
40     <unit>
41       <form>ix\"{o}q,</form>
42       <gloss>mujer</gloss>
43     </unit>
44     <unit>
45       <form>ri</form>
46       <gloss>CR</gloss>
47     </unit>
48     <unit>
49       <form>x-</form>
50       <gloss>COM-</gloss>
51     </unit>
52     <unit>
53       <form>{\0}-</form>
54       <gloss>B3s-</gloss>
55     </unit>
56     <unit>
57       <form>kos,</form>
58       <gloss>cansar</gloss>
59     </unit>
60     <unit>
61       <form>x-</form>
62       <gloss>COM-</gloss>
63     </unit>
64     <unit>
65       <form>{\0}-</form>
66       <gloss>B3s-</gloss>
67     </unit>
68     <unit>
69       <form>b'e-</form>
70       <gloss>MOV-</gloss>
71     </unit>
72     <unit>
73       <form>ru-</form>
74       <gloss>A3s-</gloss>
75     </unit>

```

```

74         <unit>
75             <form>loq'-</form>
76             <gloss>comprar-</gloss>
77         </unit>
78         <unit>
79             <form>o'</form>
80             <gloss>SC</gloss>
81         </unit>
82         <unit>
83             <form>xkoya'.</form>
84             <gloss>tomate</gloss>
85         </unit>
86     </morpheme_analysis>
87     <syntactic_analysis>
88         <unit>
89             <form>ri xkos</form>
90             <gloss>CR</gloss>
91         </unit>
92     </syntactic_analysis>
93 </interlinear_gloss>
94
95 <interlinear_gloss>
96     <parallel_phrase>
97         <original>Re ak'wal, re xtzaq, xoq'.</original>
98         <translation>Este niño que se cayó, lloró.</
99             translation>
100     </parallel_phrase>
101     <morpheme_analysis>
102         <unit>
103             <form>Re</form>
104             <gloss>DET</gloss>
105         </unit>
106         <unit>
107             <form>ak'wal,</form>
108             <gloss>niño,</gloss>
109         </unit>
110         <unit>
111             <form>re</form>
112             <gloss>CR</gloss>
113         </unit>
114         <unit>
115             <form>x-</form>
116             <gloss>COM-</gloss>
117         </unit>
118         <unit>
119             <form>{\0}</form>
120             <gloss>B3s-</gloss>
121         </unit>

```

```

122         <unit>
123             <form>tzaq,</form>
124             <gloss>caer</gloss>
125         </unit>
126         <unit>
127             <form>xloq'.</form>
128             <gloss>llorar</gloss>
129         </unit>
130     </morpheme_analysis>
131     <syntactic_analysis>
132         <unit>
133             <form>re xtzaq</form>
134             <gloss>CR</gloss>
135         </unit>
136     </syntactic_analysis>
137 </interlinear_gloss>
138
139
140 <interlinear_gloss>
141     <parallel_phrase>
142         <original>La k'aj\"{o}l, la xloq'o ri wexaj, xb'e
143         .</original>
144         <translation>Ese muchacho que compr\'o el pantal\'
145         on, se fue.</translation>
146     </parallel_phrase>
147     <syntactic_analysis>
148         <unit>
149             <form>la xloq'o ri wexaj</form>
150             <gloss>CR</gloss>
151         </unit>
152     </syntactic_analysis>
153 </interlinear_gloss>
154 </page>
155 </document>

```

Listado A.2: Document Type Definition

```

1 <!ELEMENT document (page+)>
2 <!--ATTLIST document-->
3
4 <!--ELEMENT page (line | heading | title | interlinear_gloss |
5     parallel_phrase)*-->
6 <!--ATTLIST page
7     number CDATA #REQUIRED-->
8
9 <!--ELEMENT heading (#PCDATA)-->
10 <!--ELEMENT title (#PCDATA)-->

```

```

10 <!--ELEMENT line (#PCDATA)-->
11
12 <!--ELEMENT interlinear_gloss (parallel_phrase?, morpheme_analysis?,
13     syntactic_analysis?)-->
14
15 <!--ELEMENT parallel_phrase (original, translation)-->
16 <!--ELEMENT original (#PCDATA)-->
17 <!--ELEMENT translation (#PCDATA)-->
18
19 <!--ELEMENT morpheme_analysis (unit+)-->
20 <!--ELEMENT syntactic_analysis (unit+)-->
21
22 <!--ELEMENT unit (form, gloss)-->
23 <!--ELEMENT form (#PCDATA)-->
24 <!--ELEMENT gloss (#PCDATA)-->
25 }

```

Listado A.3: Ejemplo del esquema XML inicial utilizado para representar glosas interlineales

```

1 <glosa_interlineal>
2   <corpus_paralelo>
3     <frase_original>k'am chi-Ø w-il-a'</frase_original>
4     <frase_español>No lo veo</frase_español>
5   </corpus_paralelo>
6
7   <glosa>
8     <columna1>
9       <fila1>k'am</fila1>
10      <fila2>NEG</fila2>
11    </columna1>
12
13    <columna2>
14      <fila1>chi-</fila1>
15      <fila2>INC-</fila2>
16    </columna2>
17
18    ...
19  </glosa>
20 </glosa_interlineal>

```